

Computer Architecture

CS 622 / EE 520

Spring 2026

Lecture 7
Shahid Masud

- Pipelining Hazards
 - Structure Hazards (Some Recap)
 - Control Hazards – related to Branching
- Branch Prediction
 - Static
 - Dynamic

Quiz 2
Next Week

Lecture 08: RISC-V Pipeline Implementation

CSCE 513 Computer Architecture

Department of Computer Science and Engineering

Yonghong Yan

yanyh@cse.sc.edu

<https://passlab.github.io/CSCE513>

2019 EE 488

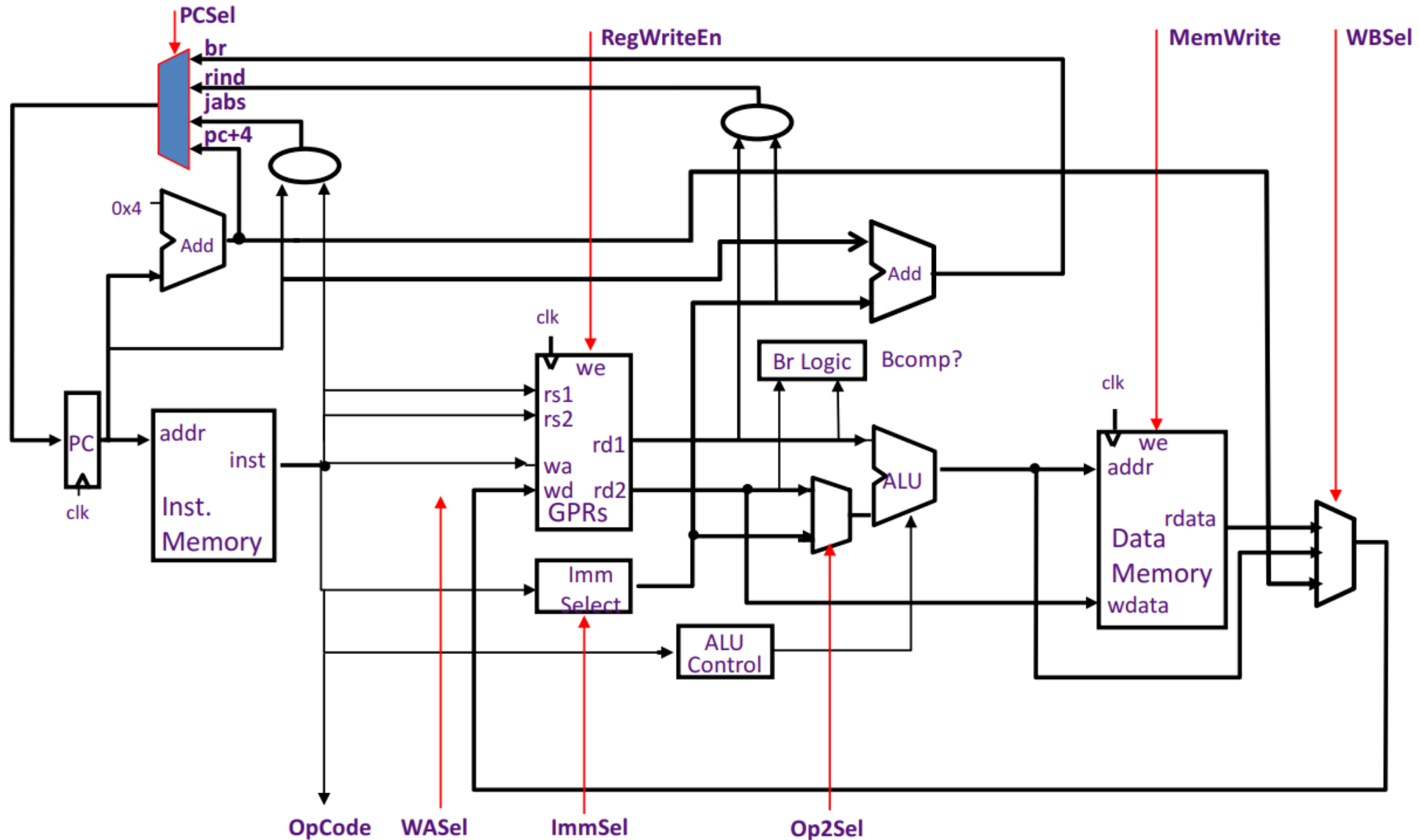
Branch Prediction

Myoungsoo Jung
Computer Division

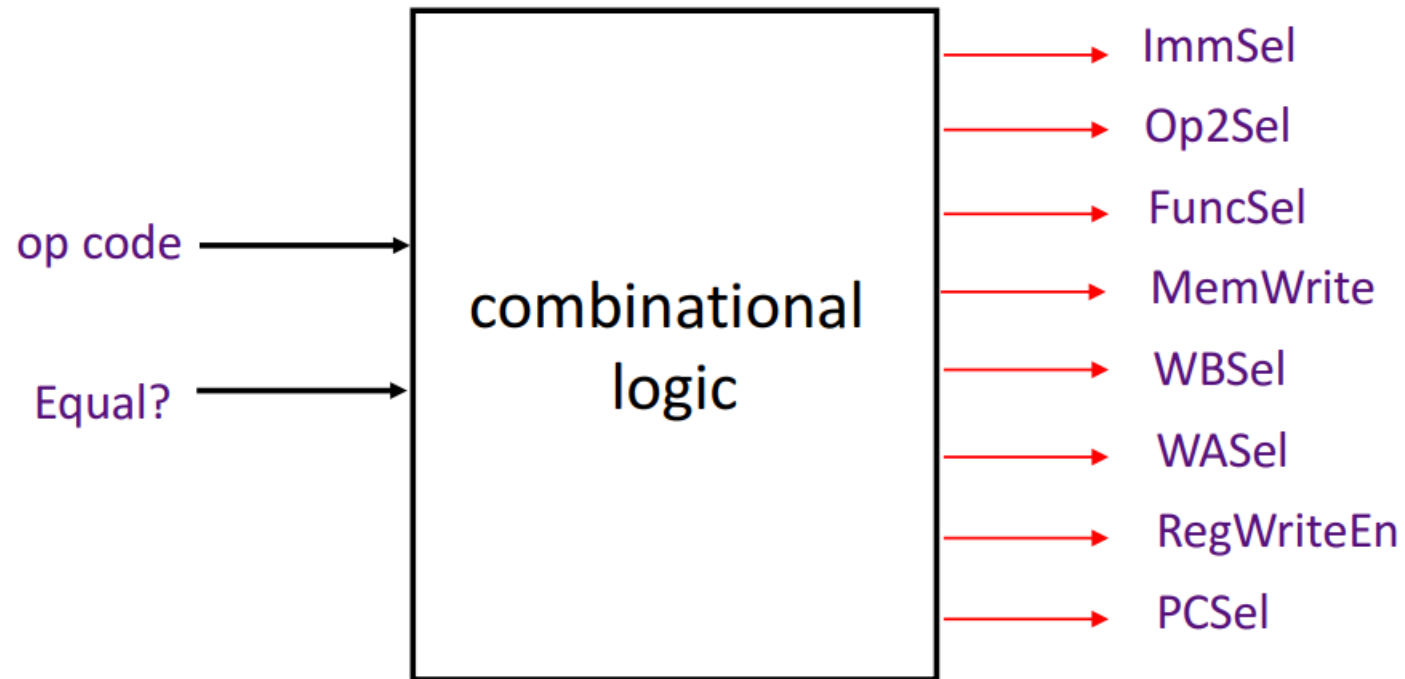
Readings:
Appendix C of course textbook

RISC V CPU – Internal View

Starting Point –Unpipelined Processor with Control



Hardwired Control Combinational Logic



Hardwired Control Table

Opcode	ImmSel	Op2Sel	FuncSel	MemWr	RFWen	WBSel	WASel	PCSel
ALU	*	Reg	Func	no	yes	ALU	rd	pc+4
ALUi	IType ₁₂	Imm	Op	no	yes	ALU	rd	pc+4
LW	IType ₁₂	Imm	+	no	yes	Mem	rd	pc+4
SW	SType ₁₂	Imm	+	yes	no	*	*	pc+4
BEQ _{true}	SByte ₁₂	*	*	no	no	*	*	br
BEQ _{false}	SByte ₁₂	*	*	no	no	*	*	pc+4
J	*	*	*	no	no	*	*	jabs
JAL	*	*	*	no	yes	PC	X1	jabs
JALR	*	*	*	no	yes	PC	rd	rind

Op2Sel= Reg / Imm

WBSel = ALU / Mem / PC

WASel = rd / X1

PCSel = pc+4 / br / rind / jabs

Example — MIPS Pipeline

■ Instruction fetch

- Increment instruction counter as part of this stage

■ Instruction decode and Register fetch

- I.e., read operand(s) from register(s)

■ Execution *or* Address Calculation

- Arithmetic operation using fetched registers or immediate operand

■ Memory access

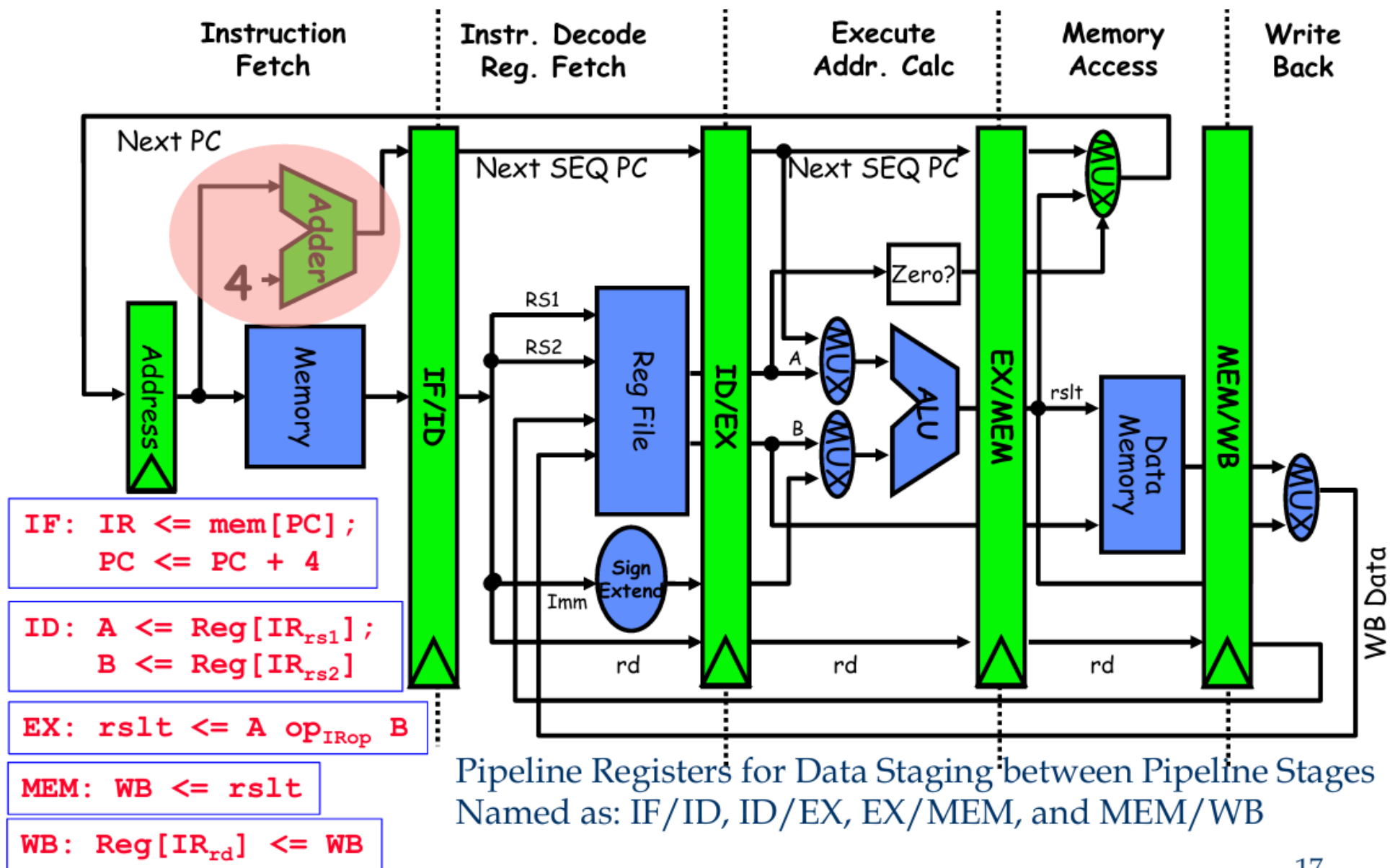
- Read from or write to (main) memory

■ Write-back to Register

- Store result into register again

Every architecture partitions its instruction set differently

Main Operations in Each Pipeline Stage

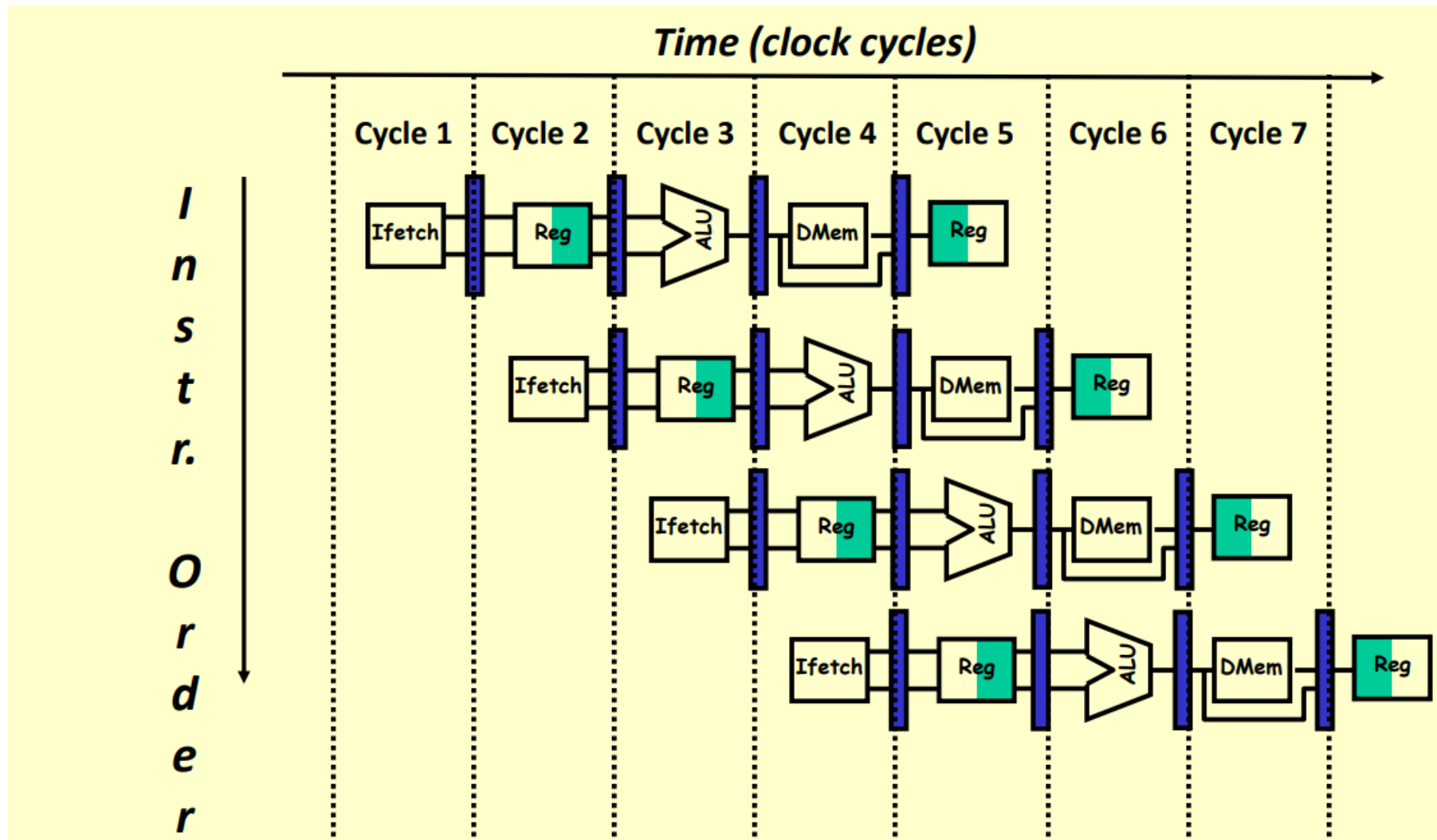


Detailed Operations in Pipeline Stages - Logic

Stage	Any Instruction		
IF	$IF/ID.IR \leftarrow MEM[PC]; IF/ID.NPC \leftarrow PC+4$ $PC \leftarrow \text{if } ((EX/MEM.opcode=branch) \ \& \ EX/MEM.cond) \{EX/MEM.ALUoutput\} \text{ else } \{PC + 4\}$		
ID	$ID/EX.A \leftarrow Regs[IF/ID.IR[Rs1]]; ID/EX.B \leftarrow Regs[IF/ID.IR[Rs2]]$ $ID/EX.NPC \leftarrow IF/ID.NPC; ID/EX.Imm \leftarrow \text{extend}(IF/ID.IR[Imm]); ID/EX.Rw \leftarrow IF/ID.IR[Rs2 \text{ or } Rd]$		
	ALU Instruction	Load / Store	Branch
EX	$EX/MEM.ALUoutput \leftarrow ID/EX.A \text{ func } ID/EX.B, \text{ or}$ $EX/MEM.ALUoutput \leftarrow ID/EX.A \text{ op } ID/EX.Imm$	$EX/MEM.ALUoutput \leftarrow ID/EX.A + ID/EX.Imm$ $EX/MEM.B \leftarrow ID/EX.B$	$EX/MEM.ALUoutput \leftarrow ID/EX.NPC + (ID/EX.Imm \ll 2)$ $EX/MEM.cond \leftarrow \text{br condition}$
MEM	$MEM/WB.ALUoutput \leftarrow EX/MEM.ALUoutput$	$MEM/WB.LMD \leftarrow MEM[EX/MEM.ALUoutput]$ or $MEM[EX/MEM.ALUoutput] \leftarrow EX/MEM.B$	
WB	$Regs[MEM/WB.Rw] \leftarrow MEM/WB.ALUOutput$	For load only: $Regs[MEM/WB.Rw] \leftarrow MEM/WB.LMD$	

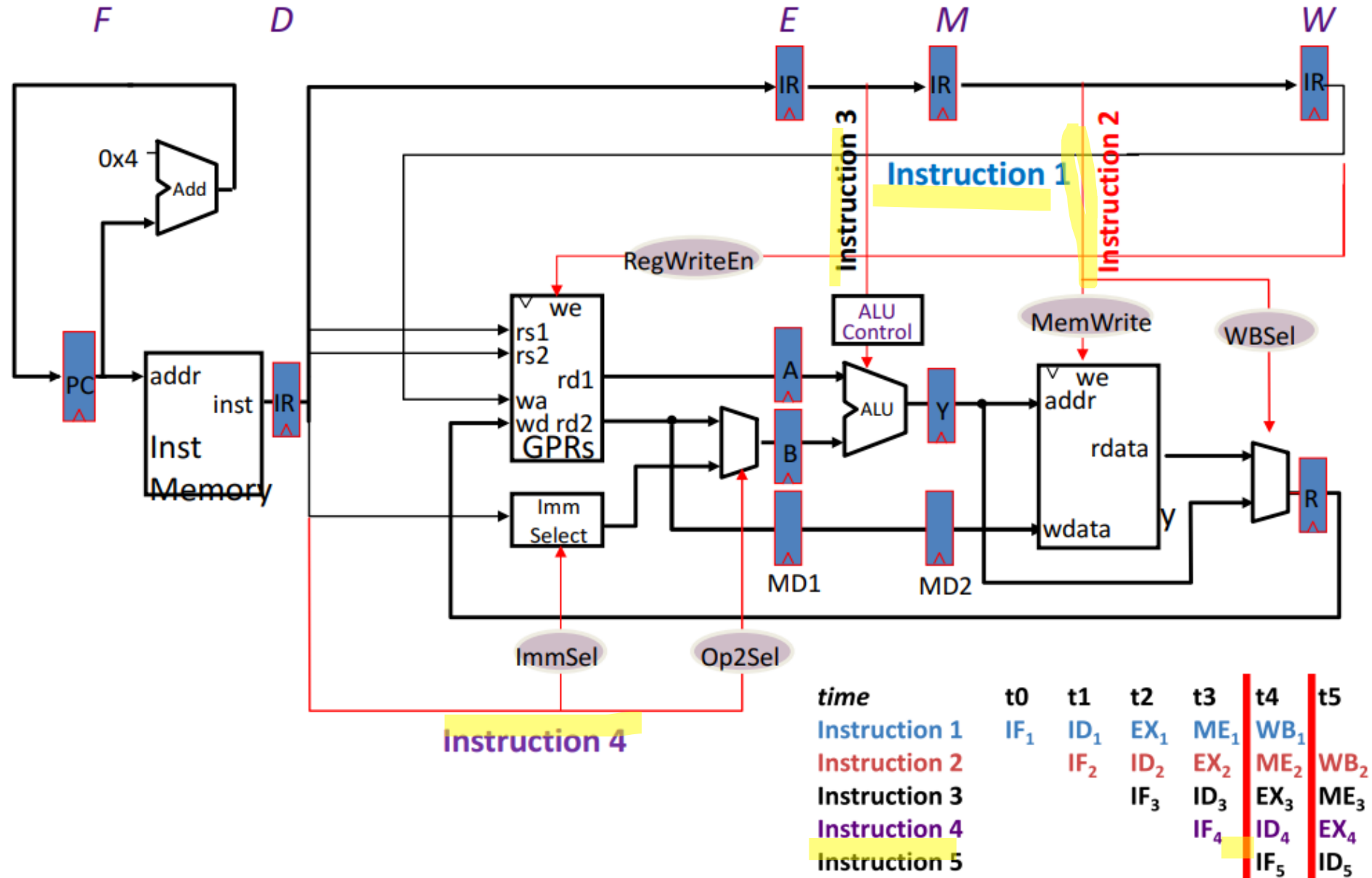
Branch requires 3 cycles;
Store requires 4 cycles, and
all other instructions require 5 cycles.

Visualizing Pipelining in a Diagram



Pipeline Hazards (Recap)

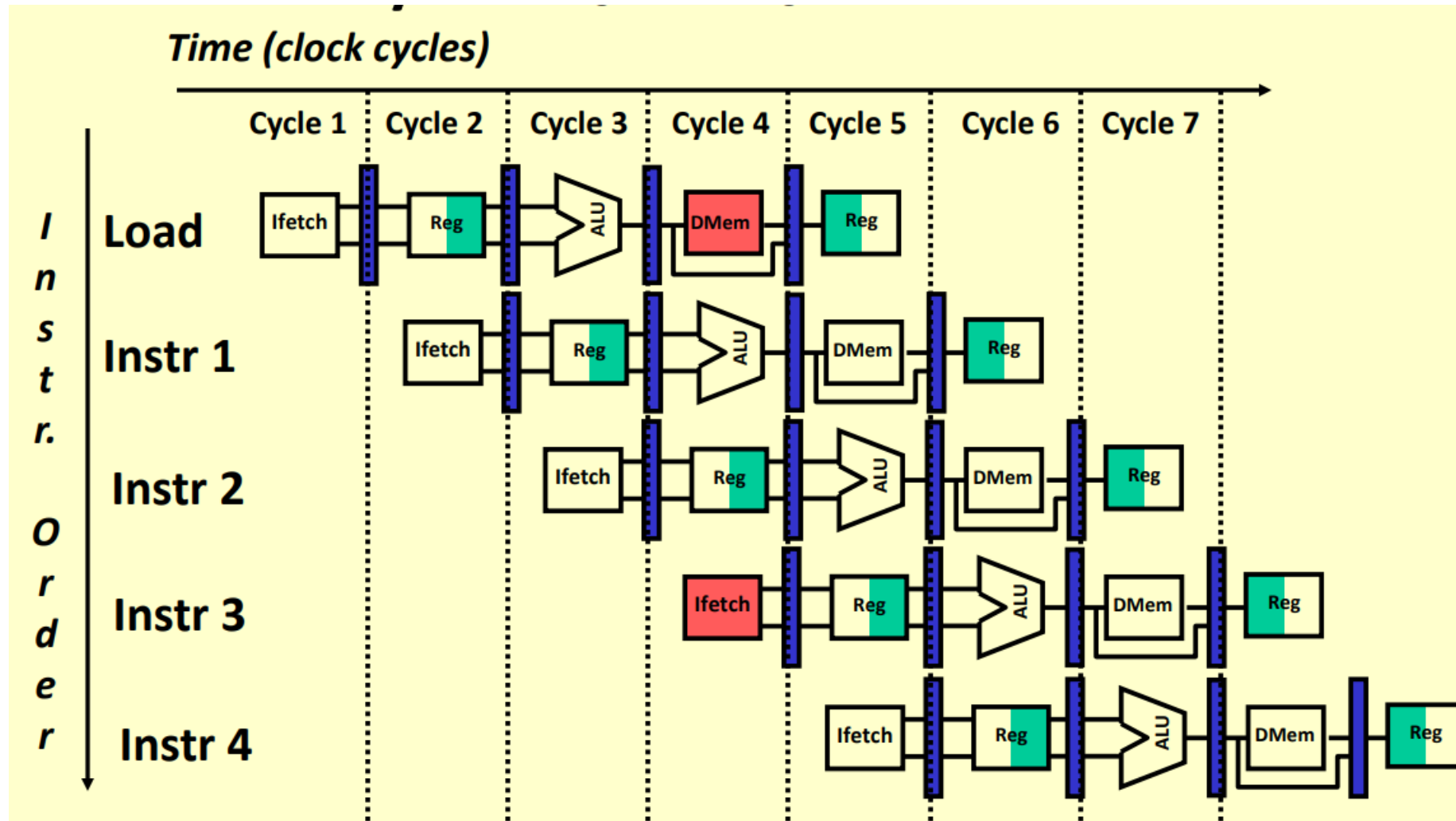
Multiple Instructions Interact inside Pipeline



Hazards

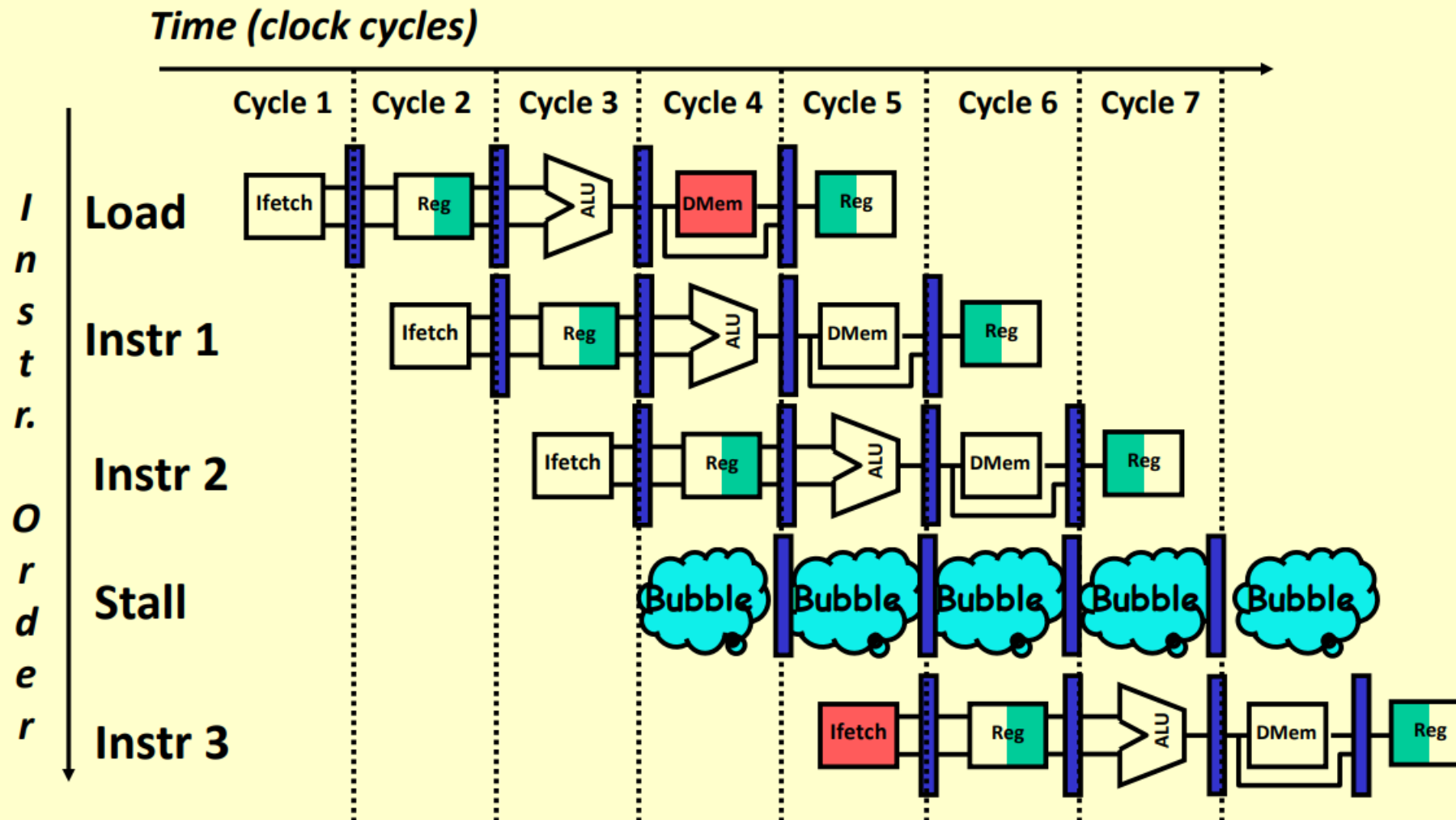
- **Structural** – resource conflicts among successive instructions
 - Hardware cannot support all possible combinations of instructions in rapid succession
- **Data** – dependencies of instructions on results of other instructions
 - $x = a * b + c$ Cannot add until multiply is done
- **Control** – branches that change instruction fetch order
 - Will affect instructions that have already been fetched!

Structural Hazard due to Memory Access



Penalty – a Bubble in Pipeline

(Similar to Figure C.5, Page C-15)



The load instruction has a delay or latency that cannot be eliminated by forwarding alone. Instead, we need to add hardware, called a *pipeline interlock*, to preserve the correct execution pattern. In general, a pipeline interlock detects a hazard and stalls the pipeline until the hazard is cleared. In this case, the interlock stalls the pipeline, beginning with the instruction that wants to use the data until the source instruction produces it. This pipeline interlock introduces a stall or bubble, just as it did for the structural hazard. The CPI for the stalled instruction increases by the length of the stall (1 clock cycle in this case).

Hazard Detection (Stalling) Unit



- Stalling unit must
 - detect if there is a data hazard
 - insert a "nop" instruction into pipeline
- Relevant information for decision
 - identify of input registers used in instruction currently in ID (either first or second operand)
 - identity of load register used in instruction currently in EX
 - control flag that there is indeed a memory read in EX
- Format of decision
 - overwrite instruction currently in ID with "nop"
 - reset program counter

Stalling Logic

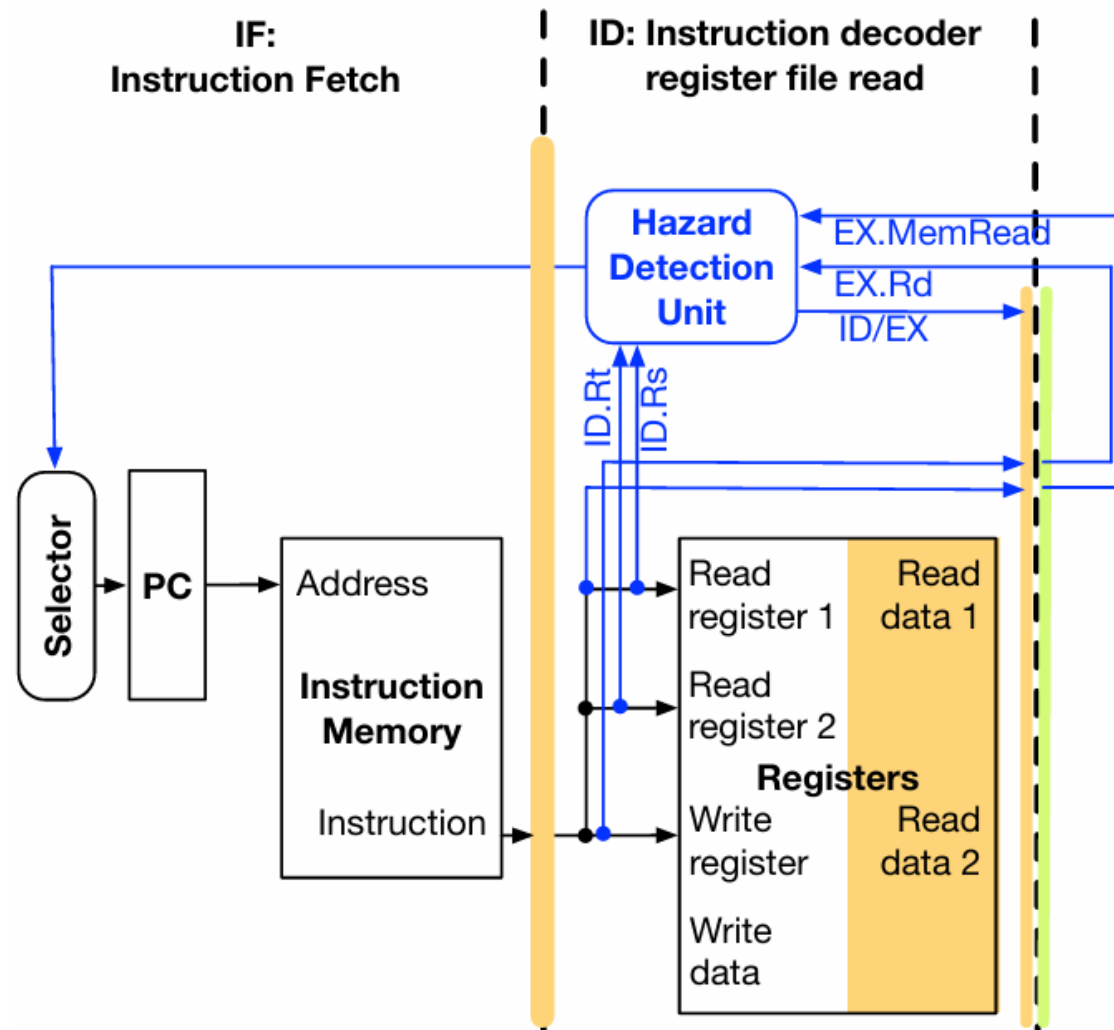
- Logic in stalling unit

```
if (EX.MemRead and  
    (EX.Rd = ID.Rs or  
     EX.Rd = ID.Rt))
```

```
  PC = PC - 4
```

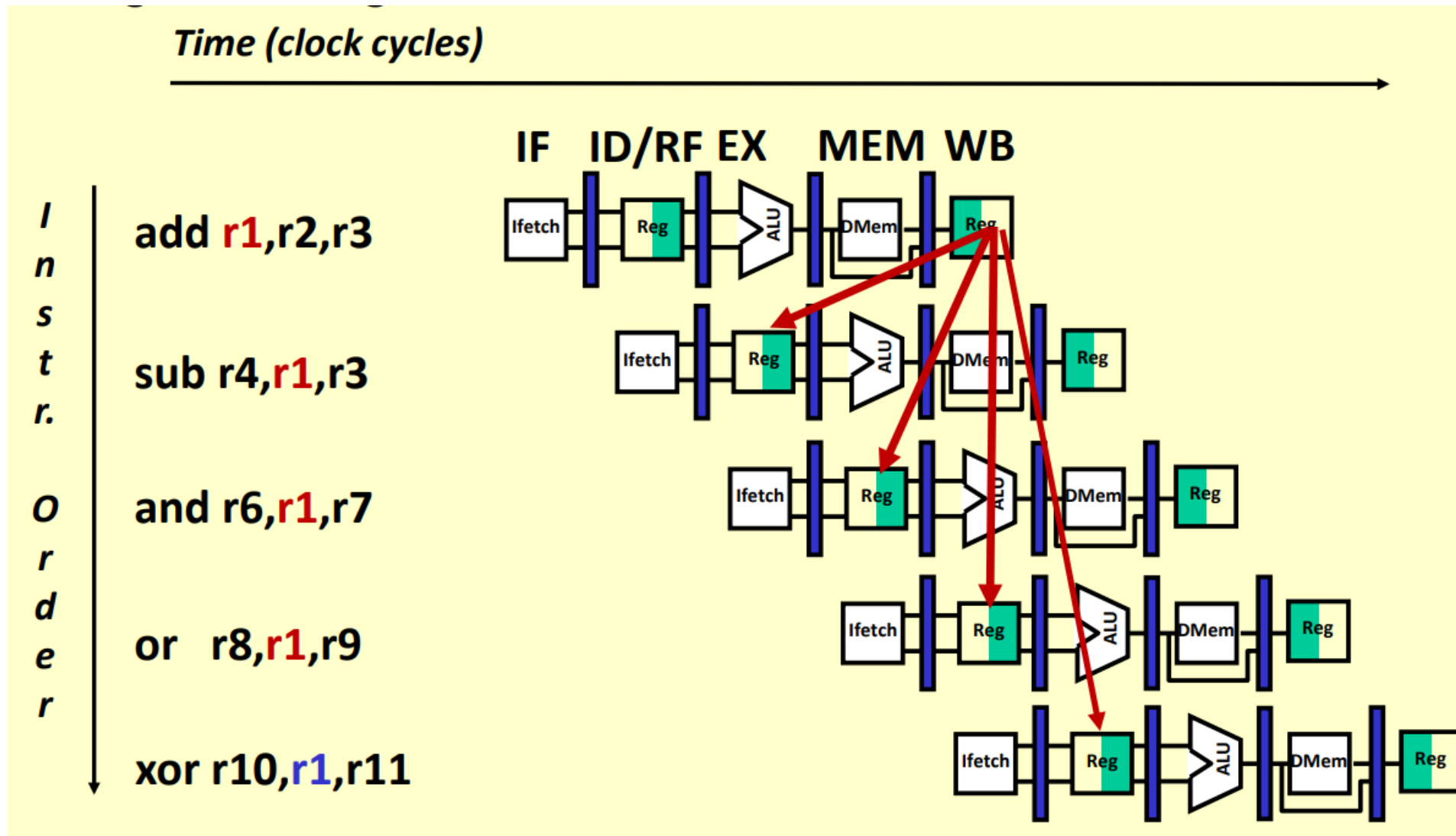
```
  ID/EX = nop
```

Stalling Unit



Forwarding to Minimize Data Hazard in Pipelining

Data Related Pipeline Hazard on R1



Forwarding Logic

15

- Logic in forwarding unit

```
if (MEM.Rd == EX.Rs)
    Forward.Rs = MEM.RdValue
    Hazard.Rs = 1
else
    Hazard.Rs = 0
```

```
if (MEM.Rd == EX.Rt)
    Forward.Rt = MEM.RdValue
    Hazard.Rt = 1
else
    Hazard.Rt = 0
```

- Must also check if "RegisterWrite" for instruction in MEM stage
- Relevant information must be passed through stages

Forwarding Conditions

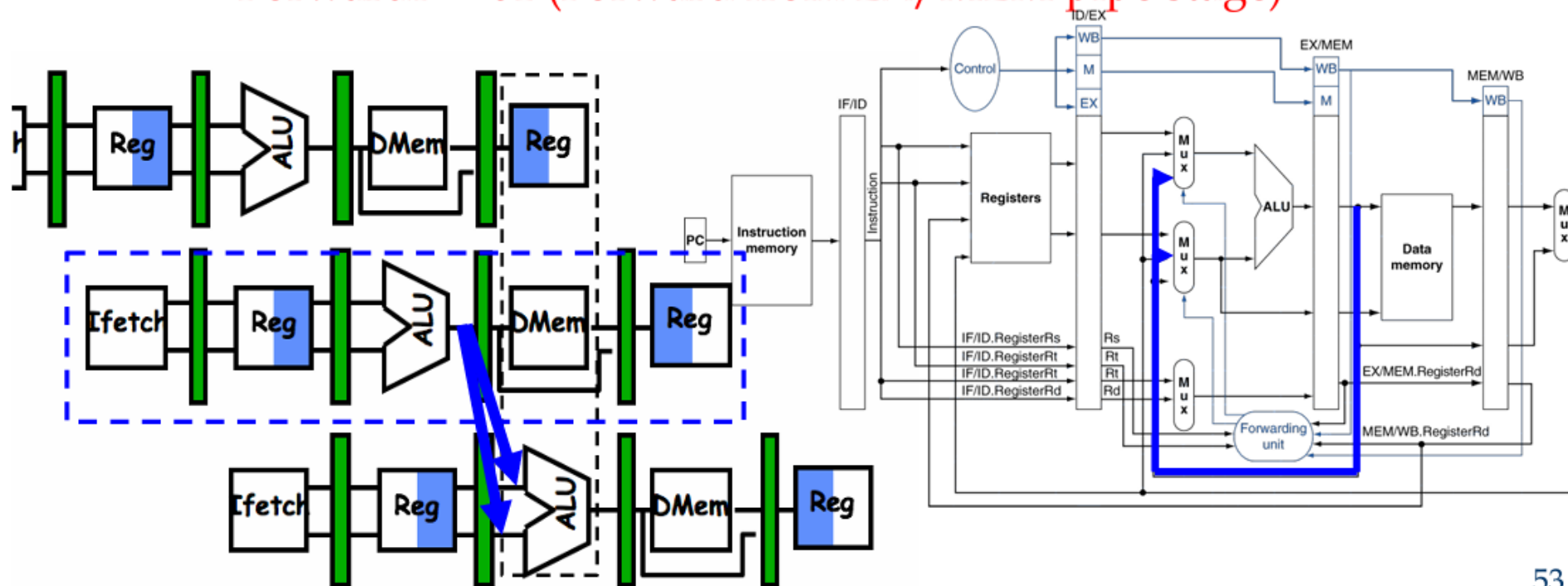
□ Detecting RAW hazard with Previous Instruction

- ◆ if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0) and (EX/MEM.RegisterRd = ID/EX.RegisterRs))

ForwardA = 01 (Forward from EX/MEM pipe stage)

- ◆ if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0) and (EX/MEM.RegisterRd = ID/EX.RegisterRt))

ForwardB = 01 (Forward from EX/MEM pipe stage)



Forwarding Condition - I

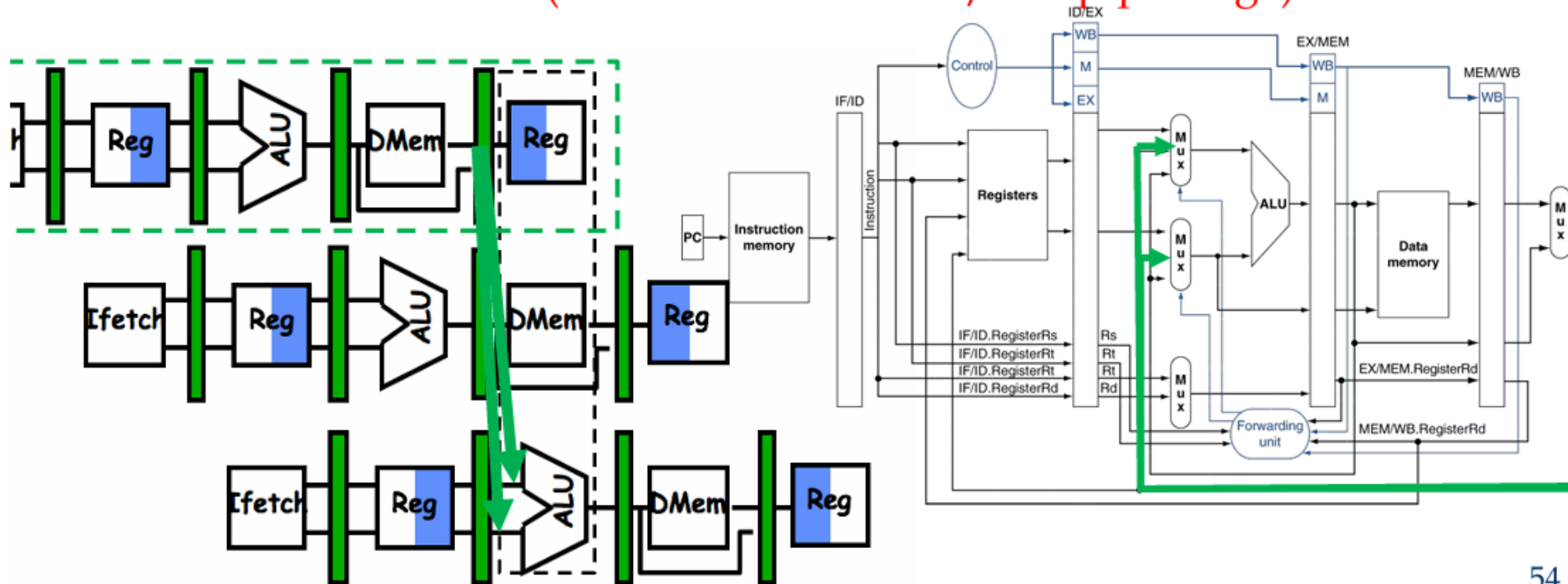
□ Detecting RAW hazard with Second Previous

- ◆ if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0) and (MEM/WB.RegisterRd = ID/EX.RegisterRs))

ForwardA = 10 (Forward from MEM/WB pipe stage)

- ◆ if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0) and (MEM/WB.RegisterRd = ID/EX.RegisterRt))

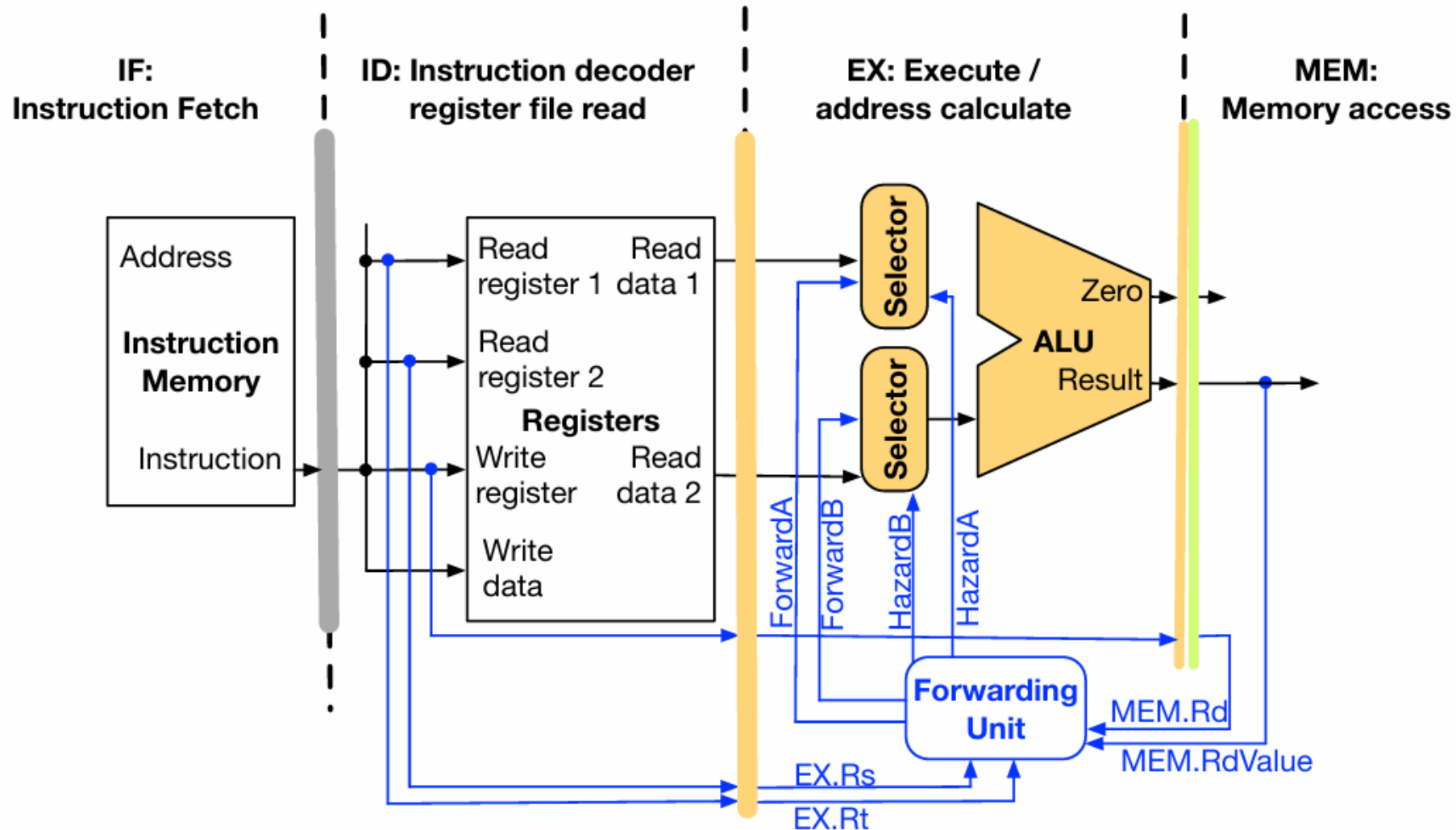
ForwardB = 10 (Forward from MEM/WB pipe stage)



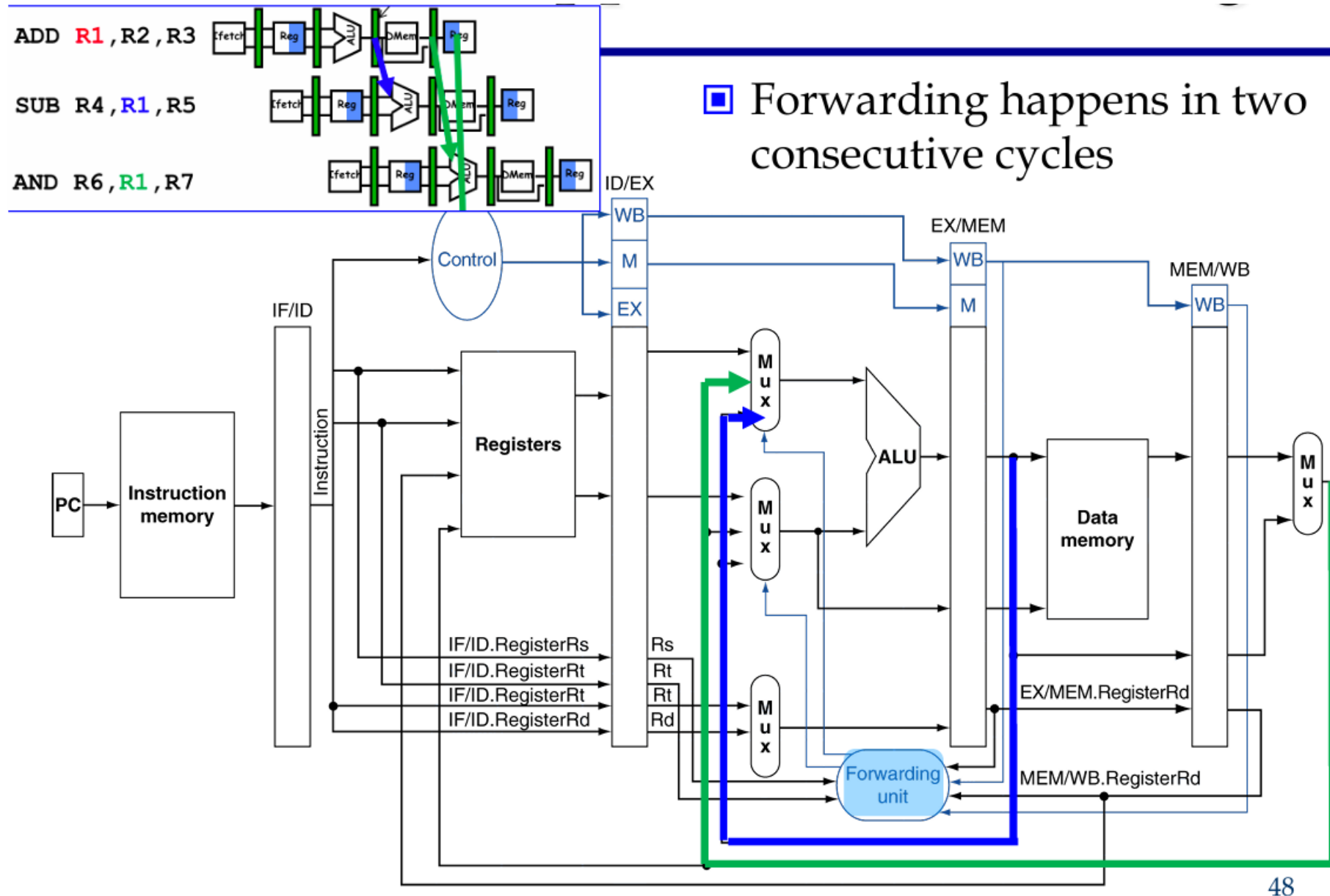
Forwarding Unit in CPU Hardware

Forwarding Unit

16



Hardware Support for Forwarding - another



Additional Forwarding Logic

- Additional logic in forwarding unit

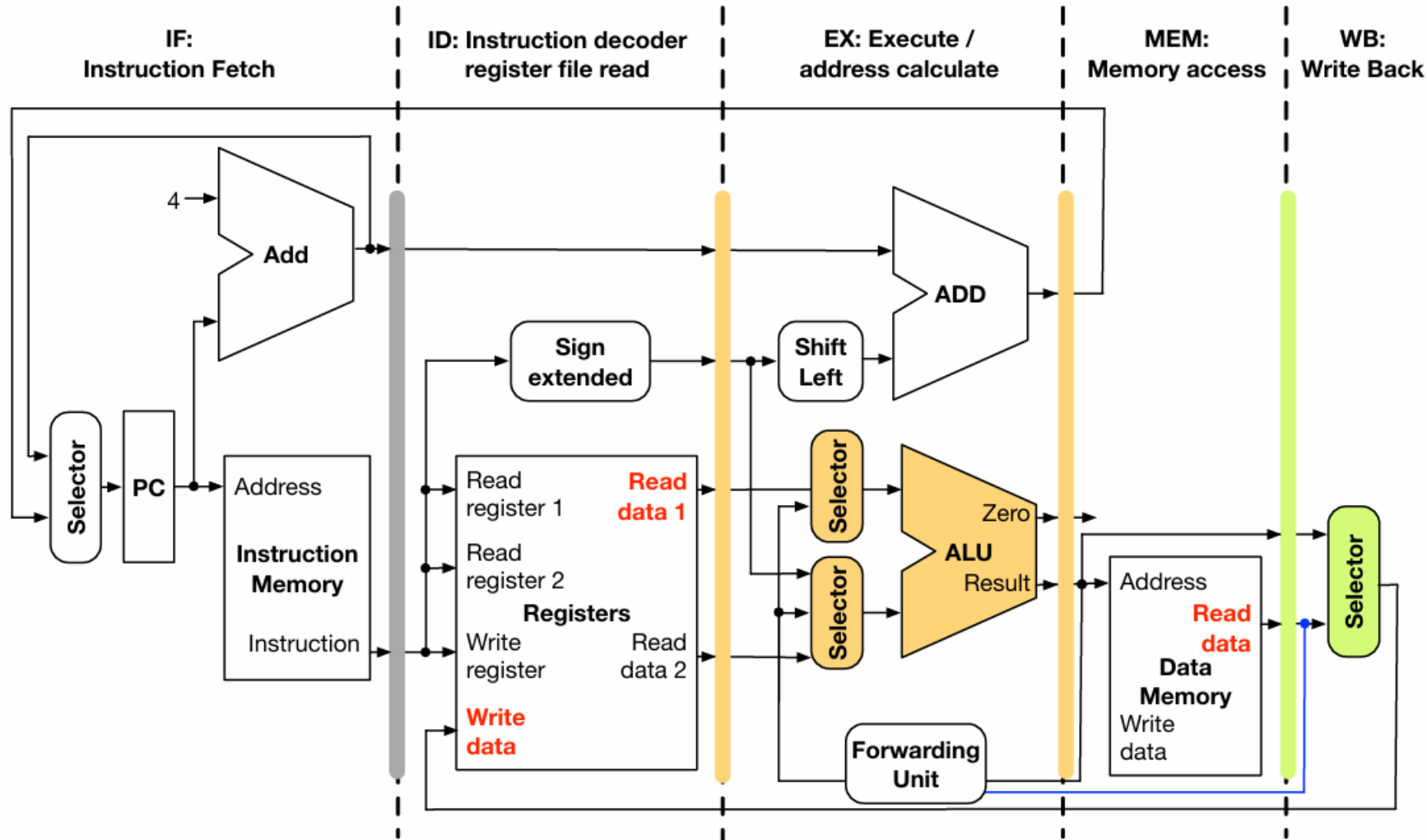
```
if (WB.Rd == EX.Rs)
    Forward.Rs = WB.RdValue
    Hazard.Rs = 1
else
    Hazard.Rs = 0
```

```
if (WB.Rd == EX.Rt)
    Forward.Rt = WB.RdValue
    Hazard.Rt = 1
else
    Hazard.Rt = 0
```

- Also relevant in "add, anything, add" sequence
where result from first add is used in last add

Forwarding Unit

38



Forwarding Considerations – Negative Time

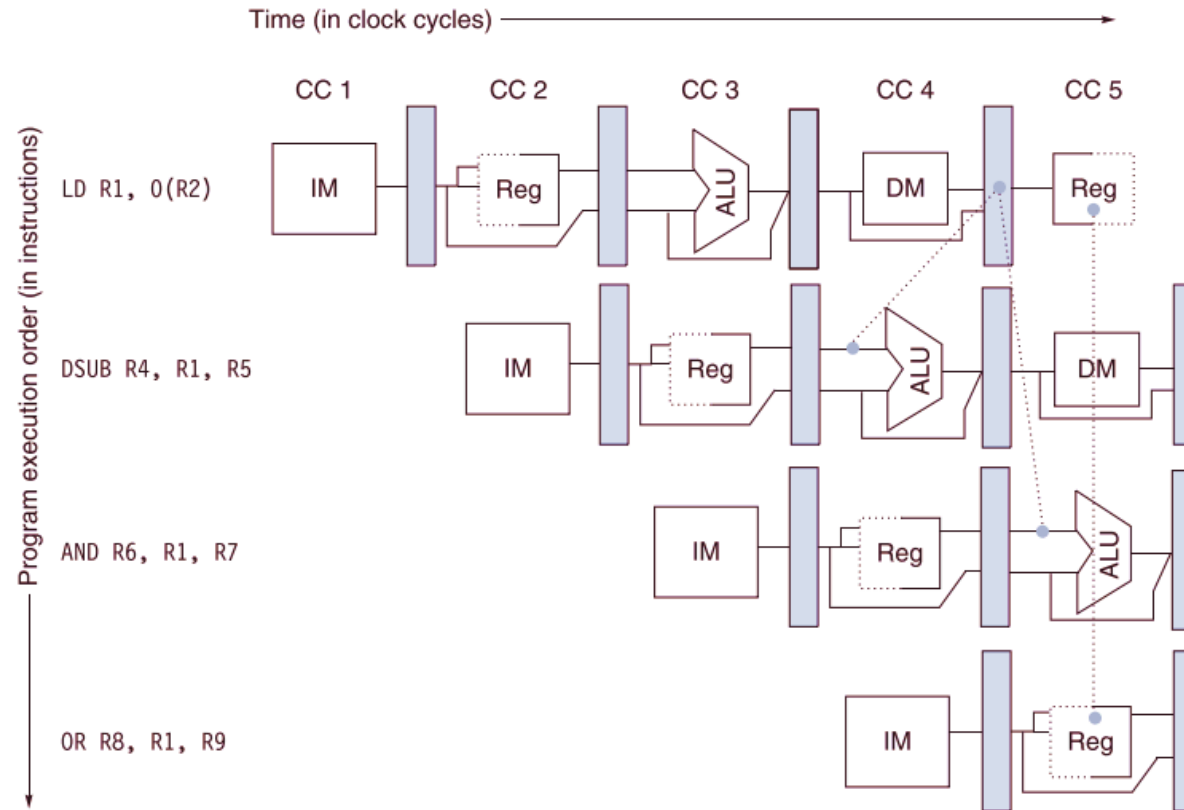


Figure C.7 The load instruction can bypass its results to the and and or instructions, but not to the sub, because that would mean forwarding the result in “negative time.”

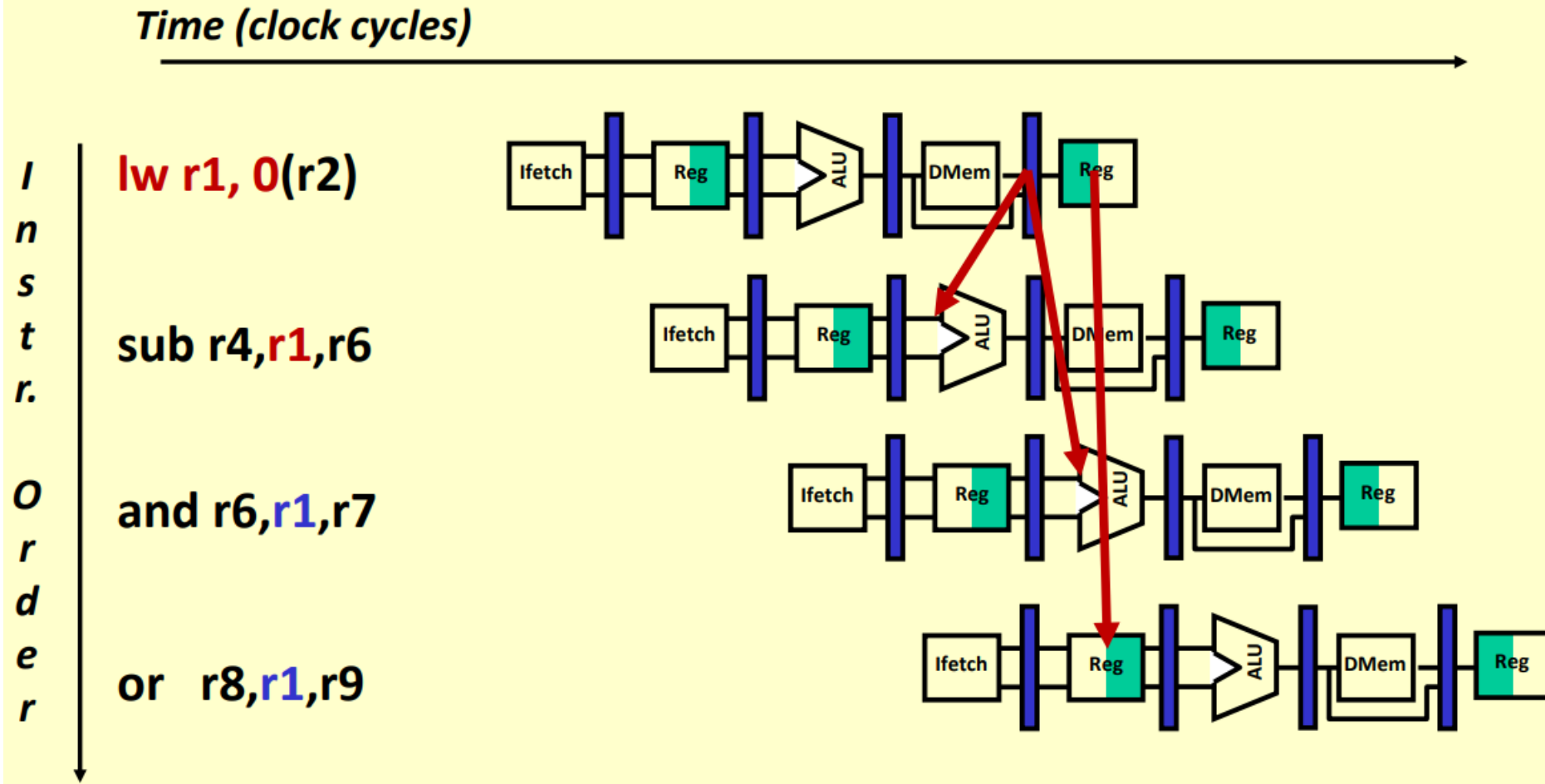
Unfortunately, not all potential data hazards can be handled by bypassing. Consider the following sequence of instructions:

```
ld    x1, 0(x2)
sub   x4, x1, x5
and   x6, x1, x7
or    x8, x1, x9
```

Hazard Possible with Forwarding

Data Hazard Even with Forwarding

(Figure C.9, Page C-20)



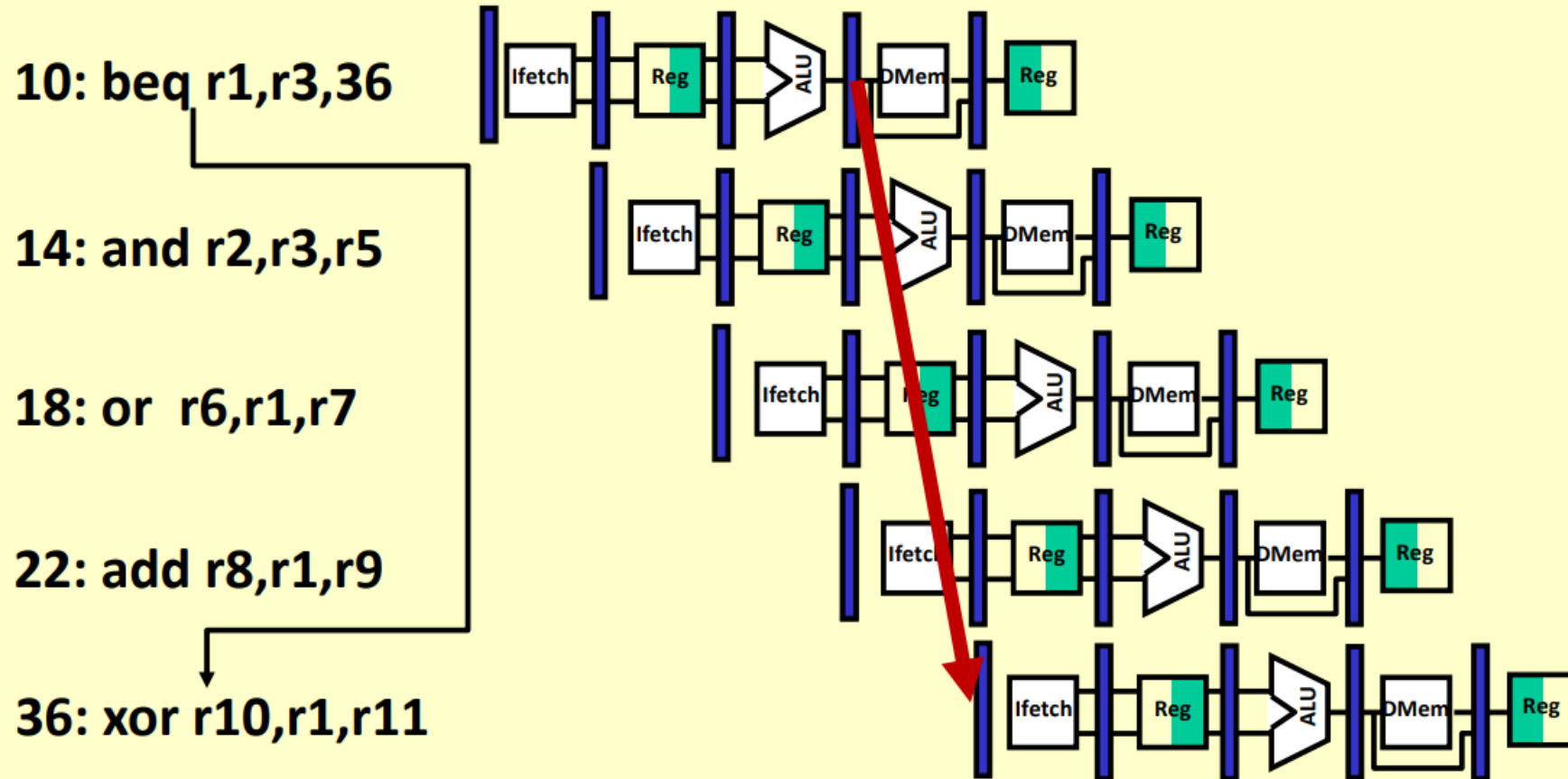
Control and Branch Hazards

Control Hazards

- Decision to take a *branch* or not may *not be known* until after the next instructions are fetched!

Control Hazard on Branches

(Three Stage Stall)



What do you do with the 3 instructions in between?

How do you do it?

Where is the “commit”?

Branch Frequency through Benchmarks

Challenge: Branches are Frequent

➤ What fraction of executed instructions are branch?

core i7; x86 ISA (SPEC2006 INT)		
Benchmark	Total Instructions	branch %
astar	5.71E+10	6.9
bzip2	4.25E+10	11.1
hmmer	2.57E+10	5.3
gcc	6.29E+09	15.1
gobmk	8.93E+10	12.1
h264	1.09E+11	7.1
libquantum	4.18E+08	13.2
omnetpp	2.55E+09	16.4
perlbench	2.91E+09	17.3
sjeng	2.11E+10	14.8
Average		9.4

Every 10th or 11th
instruction is a branch

ARM Coretex-A9; ARMv7 ISA (SPEC2006 FP)		
Benchmark	Total Instructions	branch %
bwaves	3.84E+11	13.5
cactusADM	1.02E+10	0.5
leslie3D	4.92E+10	6.2
milc	1.38E+10	6.5
tonto	1.30E+10	10.0
Average		12.15

Every 8th
instruction is a branch

Branch is Predictable in Code

Idea: But, Branch is Predictable

Program A

```
for (i=0; i<100; i++)
{
    ...
}
```

Program B

```
if (aa == 2)
    aa = 0;
if (bb == 2)
    bb = 0;
if (aa != bb)
```

There's a pattern in the original code 😊

Assembly of Program A

```
addi r10, r0, 100
addi r1, r0, r0

L1:
    ...
    addi r1, r1, 1
    bne r1, r10, L1
    ...
```

Keep this branch to L1 until i < 100

Assembly of Program B

```
addi r2, r0, 2
bne r10, r2, L_bb
xor r10, r10, r10

L_bb:
    bne r11, r2, L_xx
    xor r11, r11, r11

L_xx:
    beq r10, r11, L_exit
    ...
L_exit:
```

Branch if the previous two branches didn't take

It looks like difficult to predict the branch direction 😞

- ▣ Branch instructions can cause great performance loss

- ▣ Branch instructions need two things:

- ◆ **Branch Result** Taken or Not Taken **Direction**

- ◆ **Branch Target**
 - » $PC + 4$ If Branch is NOT taken
 - » $PC + 4 + 4 \times imm$ If Branch is Taken

Target Destination

- ▣ **For our pipeline: 3-cycle branch delay**

- ◆ PC is updated 3 cycles after fetching branch instruction
 - ◆ Branch target address is calculated in the ALU stage
 - ◆ Branch result is also computed in the ALU stage
 - ◆ What to do with the next 3 instructions after branch?

Restarting Fetch in Branch Instruction

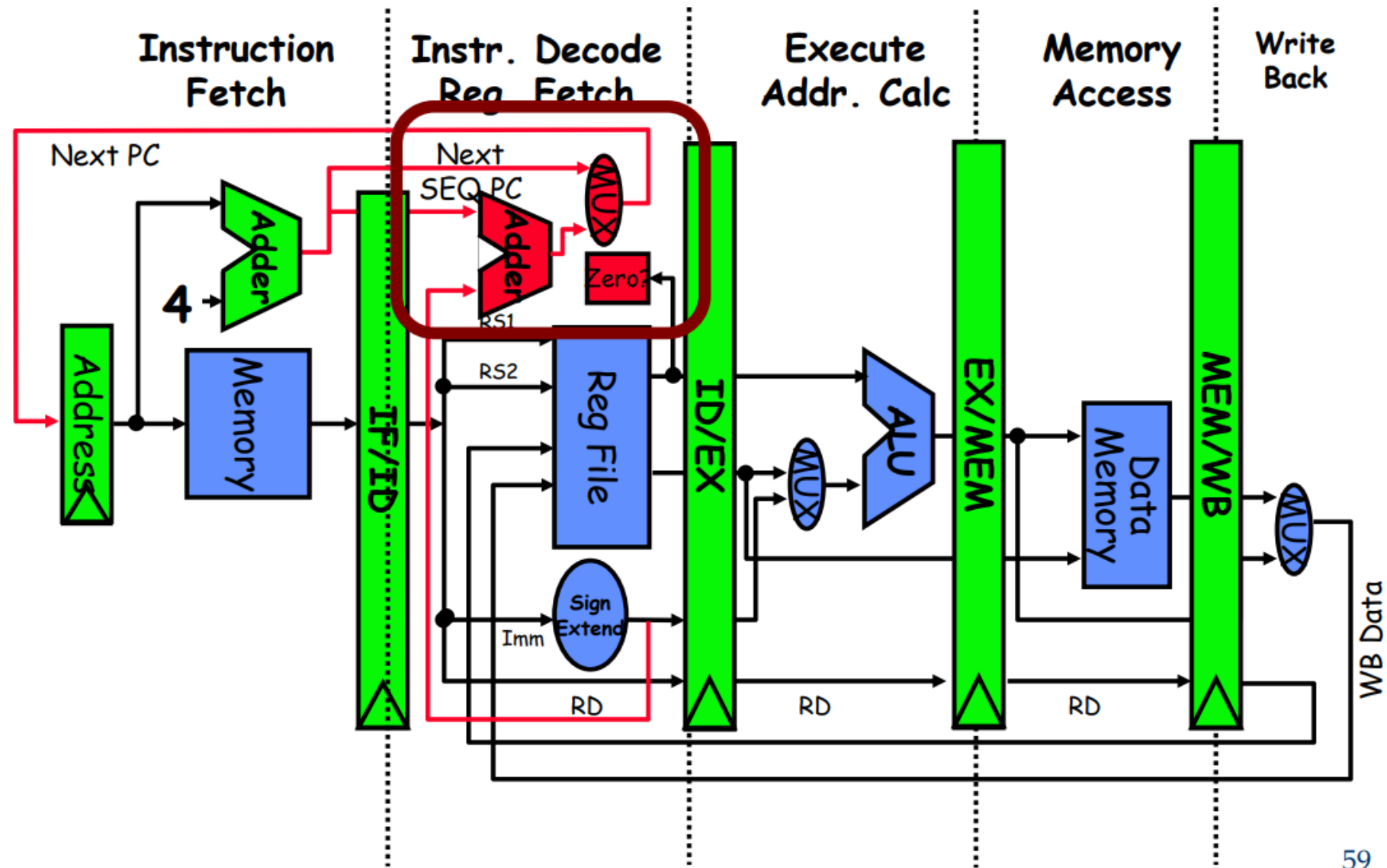
Figure C.9 shows that the simplest method of dealing with branches is to redo the fetch of the instruction following a branch, once we detect the branch during ID (when instructions are decoded). The first IF cycle is essentially a stall, because it never performs useful work. You may have noticed that if the branch is untaken, then the repetition of the IF stage is unnecessary because the correct instruction was indeed fetched. We will develop several schemes to take advantage of this fact shortly.

One stall cycle for every branch will yield a performance loss of 10% to 30% depending on the branch frequency, so we will examine some techniques to deal with this loss.

Branch instruction	IF	ID	EX	MEM	WB		
Branch successor		IF	IF	ID	EX	MEM	WB
Branch successor+ 1				IF	ID	EX	MEM
Branch successor+ 2					IF	ID	EX

Figure C.9 A branch causes a one-cycle stall in the five-stage pipeline. The instruction after the branch is fetched, but the instruction is ignored, and the fetch is restarted once the branch target is known. It is probably obvious that if the branch is not taken, the second IF for branch successor is redundant. This will be addressed shortly.

Extra Hardware for Branch in Pipelined Datapath



Branch Stall Impact

- ▣ CPI = 1 if without branch stalls, and 30% branch
- ▣ If stalling 3 cycles per branch
 - ◆ $\Rightarrow \text{new CPI} = 1 + 0.3 \times 3 = 1.9$
- ▣ Two-part solution:
 - ◆ Determine branch taken or not sooner, and
 - ◆ Compute taken branch address earlier
- ▣ MIPS Solution:
 - ◆ Move branch test to ID stage (second stage)
 - ◆ Adder to calculate new PC in ID stage
 - ◆ **Branch delay is reduced from 3 to just 1 clock cycle**

Static Branch Prediction

How to do Branch Prediction?

► **Solution:** Branch Prediction

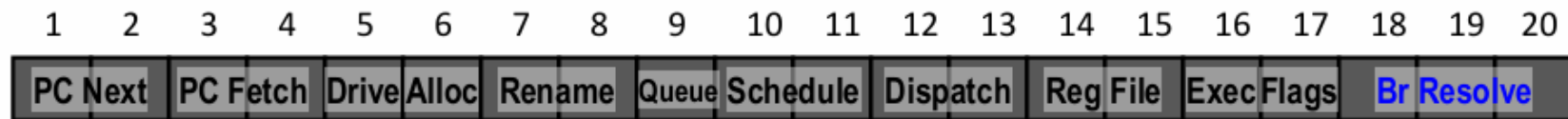
- Again, we need to know two things for branch
 - Whether the branch is taken or not (**direction**)
 - The target address if it is taken (**target**)

	Case	Direction	Target
The goal of <i>direction</i> predictors	Direct jumps, Function calls	Known (always taken)	Easy to compute
	Conditional branches (typically PC-relative)	Difficult to predict	Easy to compute
The goal of <i>target</i> predictors	Indirect jumps, Function returns	Known (always taken)	Difficult to target

What happens in Misprediction?

• **Problem:** Branch Misprediction Penalty

➤ Example: 20 pipeline stages architecture



Single Issue (flush entailed instructions and re-fetch)



Fetch the correct path



Mispredict

- ▣ #1: Stall until branch direction is clear
- ▣ #2: Predict Branch Not Taken
 - ◆ Execute successor instructions in sequence
 - ◆ “Squash” instructions in pipeline if branch actually taken
 - ◆ Advantage of late pipeline state update
 - ◆ 47% MIPS branches not taken on average
 - ◆ PC+4 already calculated, so use it to get next instruction
- ▣ #3: Predict Branch Taken
 - ◆ 53% MIPS branches taken on average
 - ◆ But haven't calculated branch target address in MIPS
 - » MIPS still incurs 1 cycle branch penalty
 - » Other machines: branch target known before outcome

▣ #4: Delayed Branch

- ◆ Define branch to take place **AFTER** a following instruction

branch instruction

sequential successor₁

sequential successor₂

.....

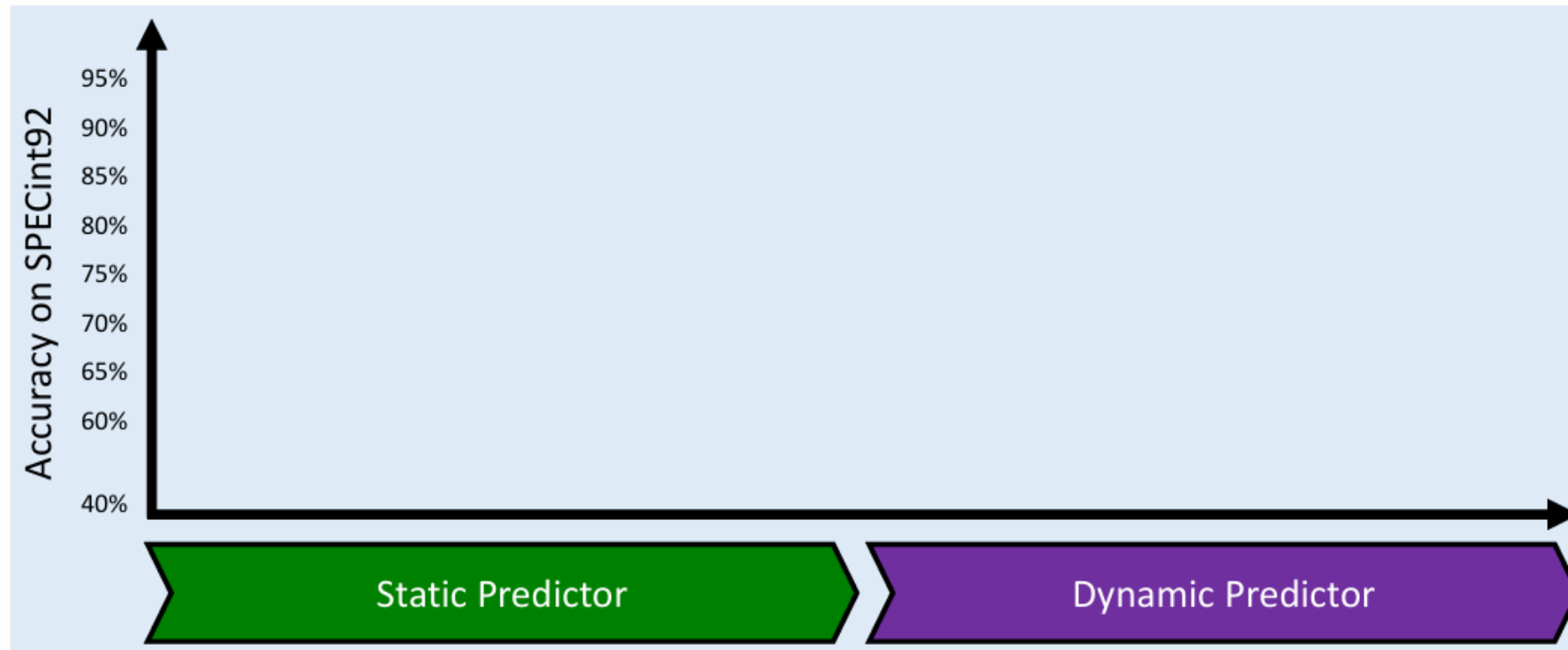
sequential successor_n

branch target if taken

- ◆ 1 slot delay allows proper decision and branch target address in 5 stage pipeline

Static Direction Prediction

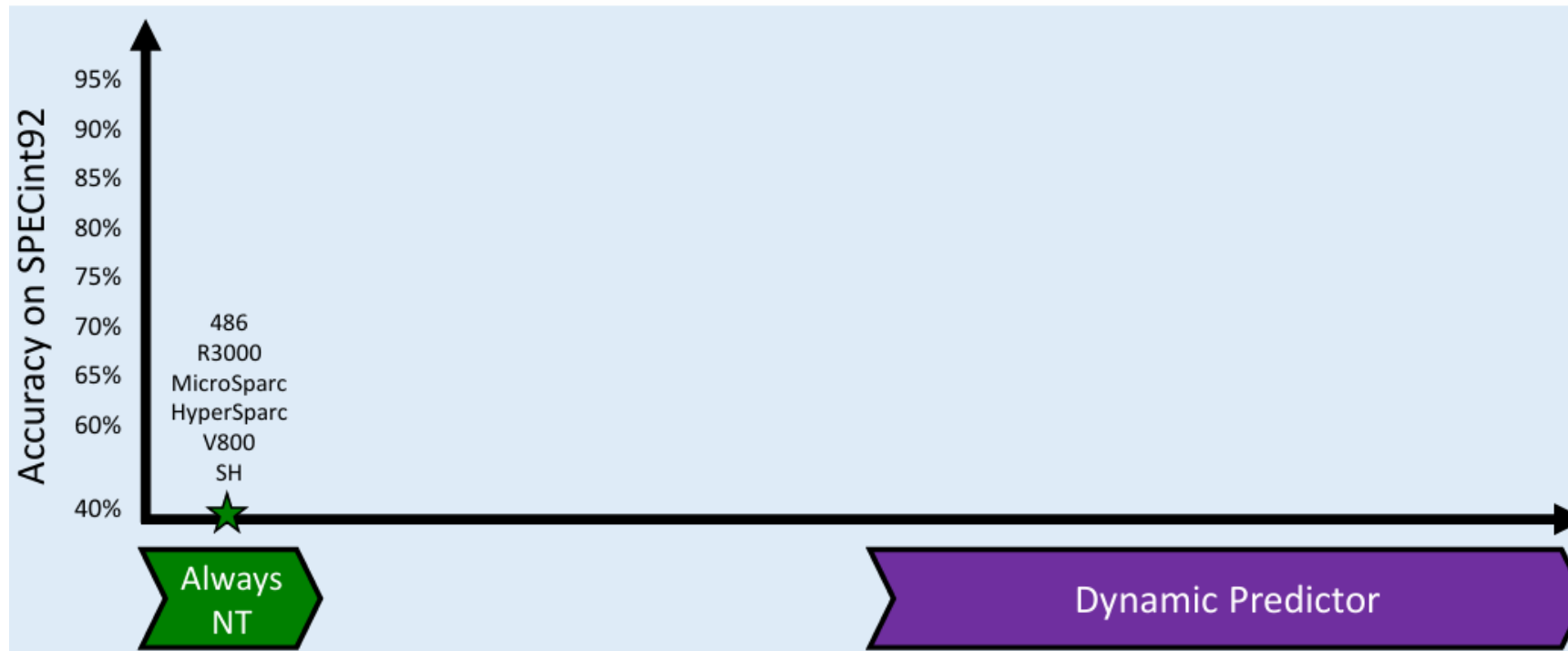
- Statically predicts the branch direction at compile time
- Old processors use this scheme because of its simplicity and low cost



➤ **Static** Direction Prediction

① Always Not Taken

- Always predict branches to be not taken
- Easy to implement, but 30-40% accuracy



➤ **Static** Direction Prediction

② Always Taken

- Always predict branches to be taken
- 60-70% accuracy (as Always NT accuracy of 30-40%)

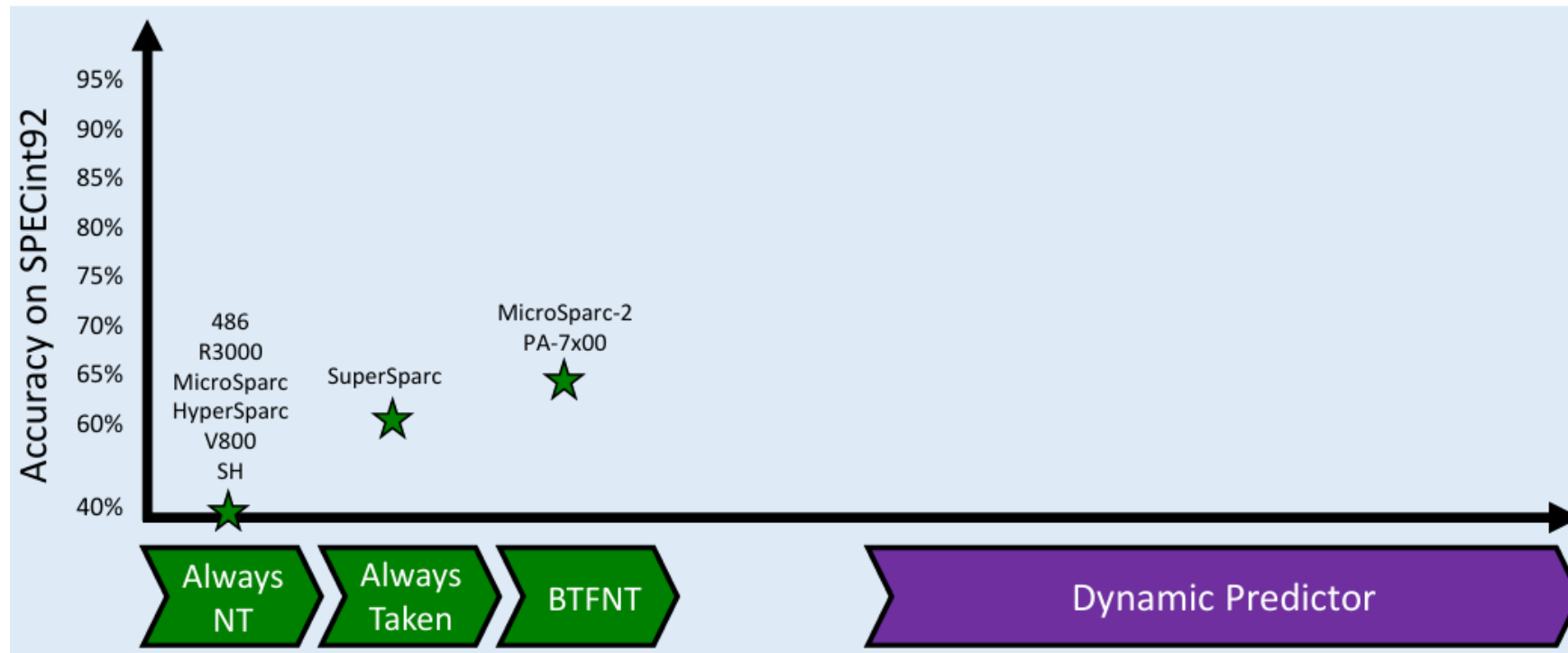


Predict Backward Taken, Forward Not Taken

➤ Static Direction Prediction

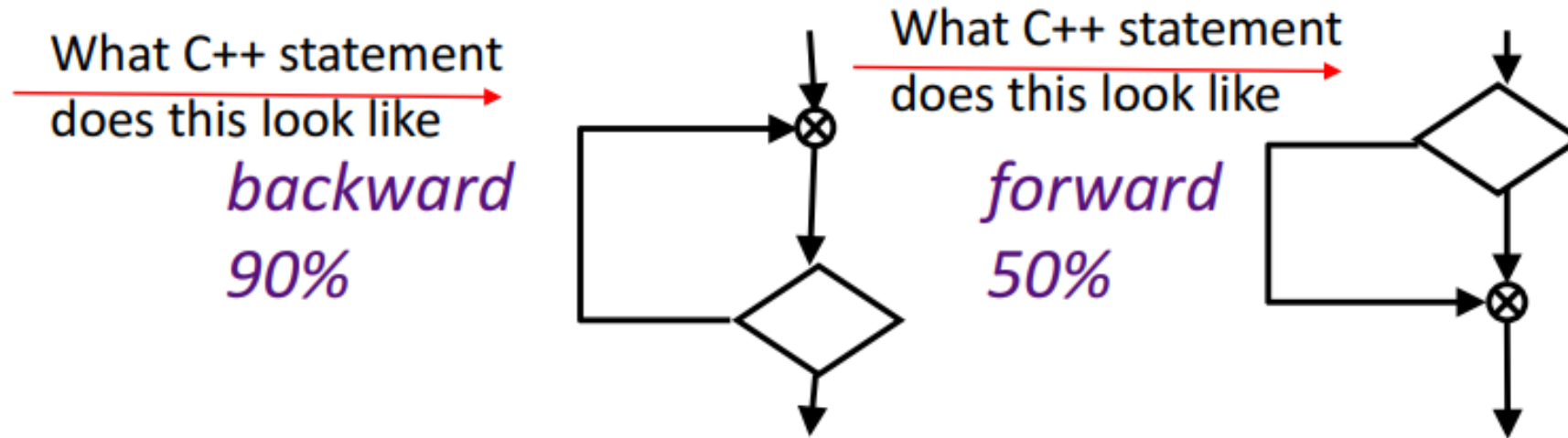
③ BTFNT (Backward Taken, Forward Not Taken)

- Always take the smaller address; advantage in **loops**
- But, doesn't work well on programs with irregular branches



Static Branch Prediction – Backward and Forward

Overall probability a branch is taken is ~60-70% but:



ISA can attach preferred direction semantics to branches, e.g.,

Motorola MC88110

bne0 (preferred taken) *beq0 (not taken)*

Predicted Taken / Not Taken – Pipeline Sequence



Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB
Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	idle	idle	idle	idle			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

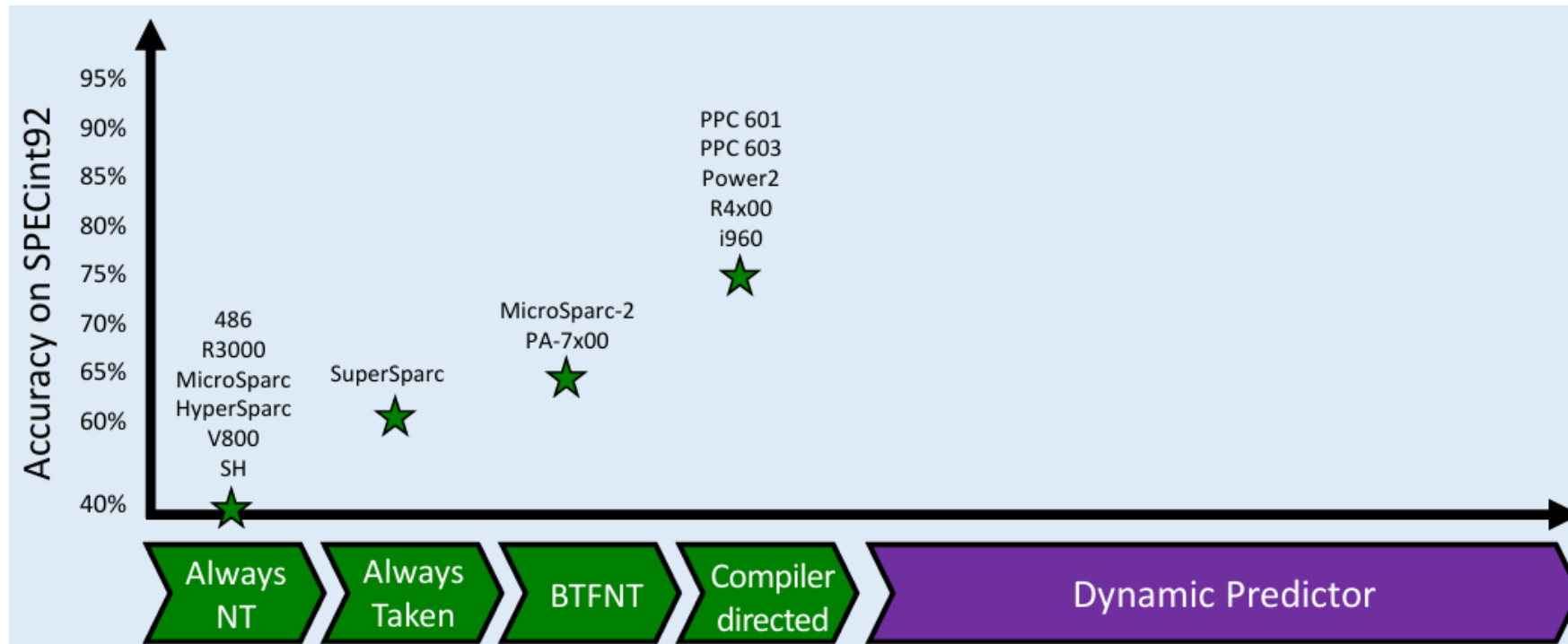
Figure C.10 The predicted-not-taken scheme and the pipeline sequence when the branch is **untaken (top)** and **taken (bottom)**. When the branch is untaken, determined during ID, we fetch the fall-through and just continue. If the branch is taken during ID, we restart the fetch at the branch target. This causes all instructions following the branch to stall 1 clock cycle.

Predict Using Compiler Profiling

➤ Static Direction Prediction

④ Compiler Directed

- Using techniques called profiling or feedback-directed compilation
- After the program is initially compiled, it runs using test data to determine the typical direction of each branch; the program is then **recompiled** to adjust the branch-prediction bits
- 75% accuracy!



Behavior of Delayed Branch

A fourth scheme, which was heavily used in early RISC processors is called *delayed branch*. In a delayed branch, the execution cycle with a branch delay of one is

```
branch instruction
sequential successor1
branch target if taken
```

The sequential successor is in the *branch delay slot*. This instruction is executed **whether or not the branch is taken**. The pipeline behavior of the five-stage pipeline with a branch delay is shown in Figure C.11. Although it is possible to have a branch delay longer than one, in practice almost all processors with delayed branch have a single instruction delay; other techniques are used if the pipeline has a longer potential branch penalty. The job of the compiler is to make the successor instructions valid and useful.

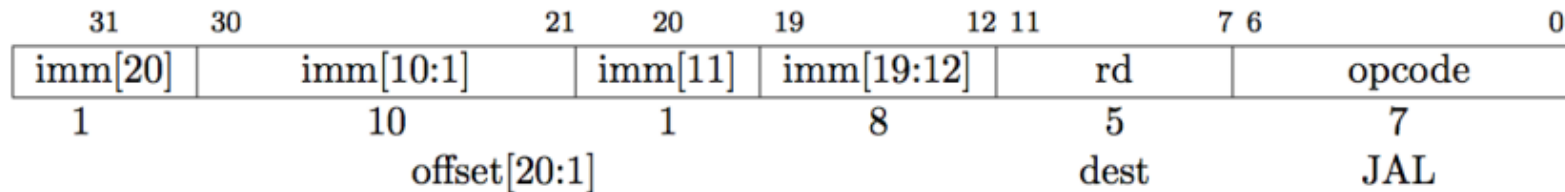
Untaken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i+1$)		IF	ID	EX	MEM	WB			
Instruction $i+2$			IF	ID	EX	MEM	WB		
Instruction $i+3$				IF	ID	EX	MEM	WB	
Instruction $i+4$					IF	ID	EX	MEM	WB
Taken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i+1$)		IF	ID	EX	MEM	WB			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

Figure C.11 The behavior of a delayed branch is the same whether or not the branch is taken. The instructions in the delay slot (there was only one delay slot for most RISC architectures that incorporated them) are executed. If the branch is untaken, execution continues with the instruction after the branch delay instruction; if the branch is taken, execution continues at the branch target. When the instruction in the branch delay slot is also a branch, the meaning is unclear: if the branch is not taken, what should happen to the branch in the branch delay slot? Because of this confusion, architectures with delay branches often disallow putting a branch in the delay slot.

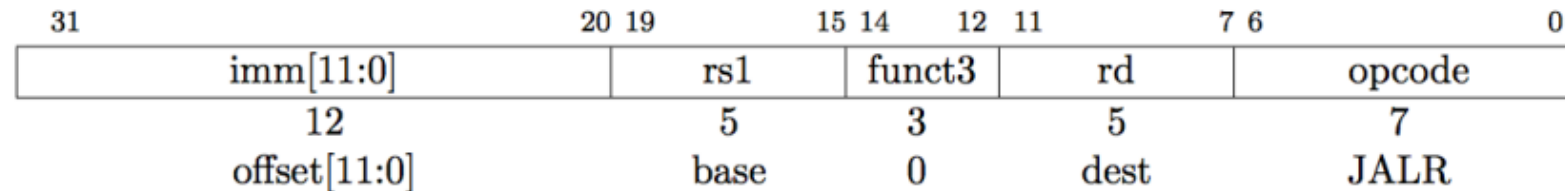
Control Instruction Details

Control Hazards: Branches and Jumps

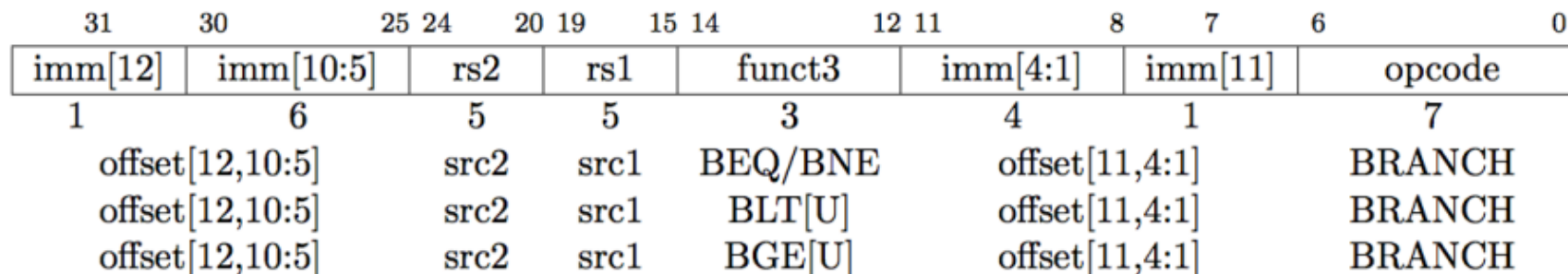
- **JAL: unconditional jump to PC+immediate**



- **JALR: indirect jump to rs1+immediate**



- **Branch: if (rs1 conds rs2), branch to PC+immediate**

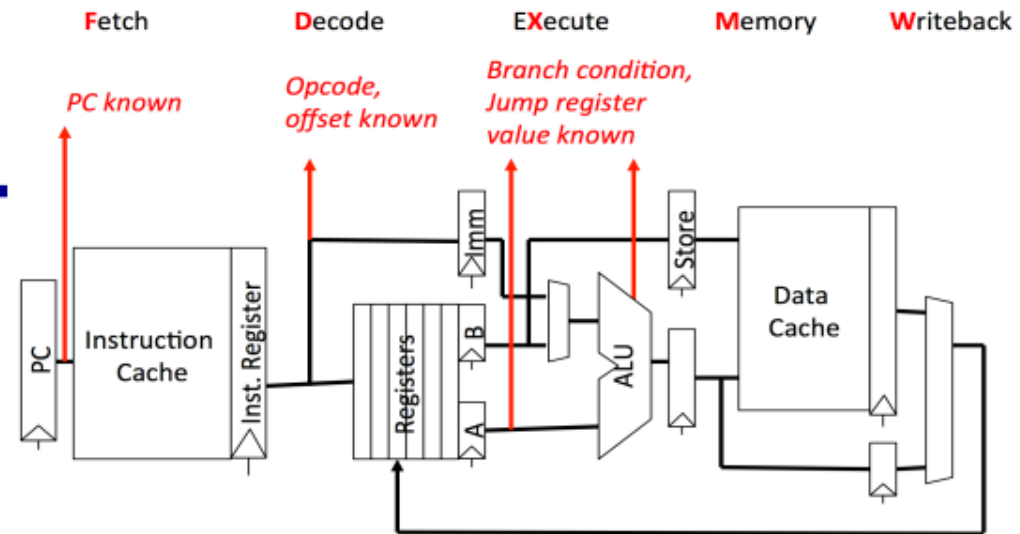


When is Destination Known?

Info for Control Transfer

Two pieces of info:

1. Taken or Not Taken
2. Target address?



- **JAL**: unconditional jump to **PC+immediate**
- **JALR**: indirect jump to **rs1+immediate**
- Branch: if (**rs1 conds rs2**), branch to **PC+immediate**

<i>Instruction</i>	<i>Taken known?</i>	<i>Target known?</i>
JAL	After Inst. Decode	After Inst. Decode
JALR	After Inst. Decode	After Reg. Fetch
B<cond.>	After Execute	After Inst. Decode

- **JAL: unconditional jump to PC+immediate**
 - 1 cycle delay of pipeline
- **JALR: indirect jump to rs1+immediate**
 - 1 cycle delay
- **Branch: if (rs1 conds rs2), branch to PC+immediate**
 - 2 cycles delay
 - 1 cycle delay for simpler branch (BEQZ) with pipeline improvement

Evaluating Branch Alternatives

- ▣ The effective pipeline speedup with branch penalties, assuming an ideal CPI of 1, is

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Pipeline stall cycles from branches}}$$

Because of the following:

$$\text{Pipeline stall cycles from branches} = \text{Branch frequency} \times \text{Branch penalty}$$

We obtain

$$\text{Pipeline speedup} = \frac{\text{Pipeline depth}}{1 + \text{Branch frequency} \times \text{Branch penalty}}$$

Performance of Control Hazard

Performance on Control Hazard (1/2)

- **Example 3** (pA-25): for a deeper pipeline, such as that in a MIPS R4000, it takes at least three pipeline stages before the branch-target address is known and an additional cycle before the branch condition is evaluated, assuming no stalls on the registers in the conditional comparison, A three-stage delay leads to the branch penalties for the three simplest prediction schemes listed in the following Figure A.15. Find the effective additional to the CPI arising from branches for this pipeline, assuming the following frequencies:

Unconditional branch	4%
Conditional branch, untaken	6%
Conditional branch, taken	10%

Figure A.15

Branch scheme	Penalty unconditional	Penalty untaken	Penalty taken
Flush pipeline	2	3	3
Predicted taken	2	3	2
Predicted un taken	2	0	3

Performance on Control Hazard (2/2)

▣ Answer

Additions to the CPI from branch cost				
Branch scheme	Unconditional branches	Untaken conditional branches	Taken conditional branches	All branches
Frequency of event	4%	6%	10%	20%
Stall pipeline	0.08	0.18	0.30	0.56
Predicted taken	0.08	0.18	0.20	0.46
Predicted untaken	0.08	0.00	0.30	0.38

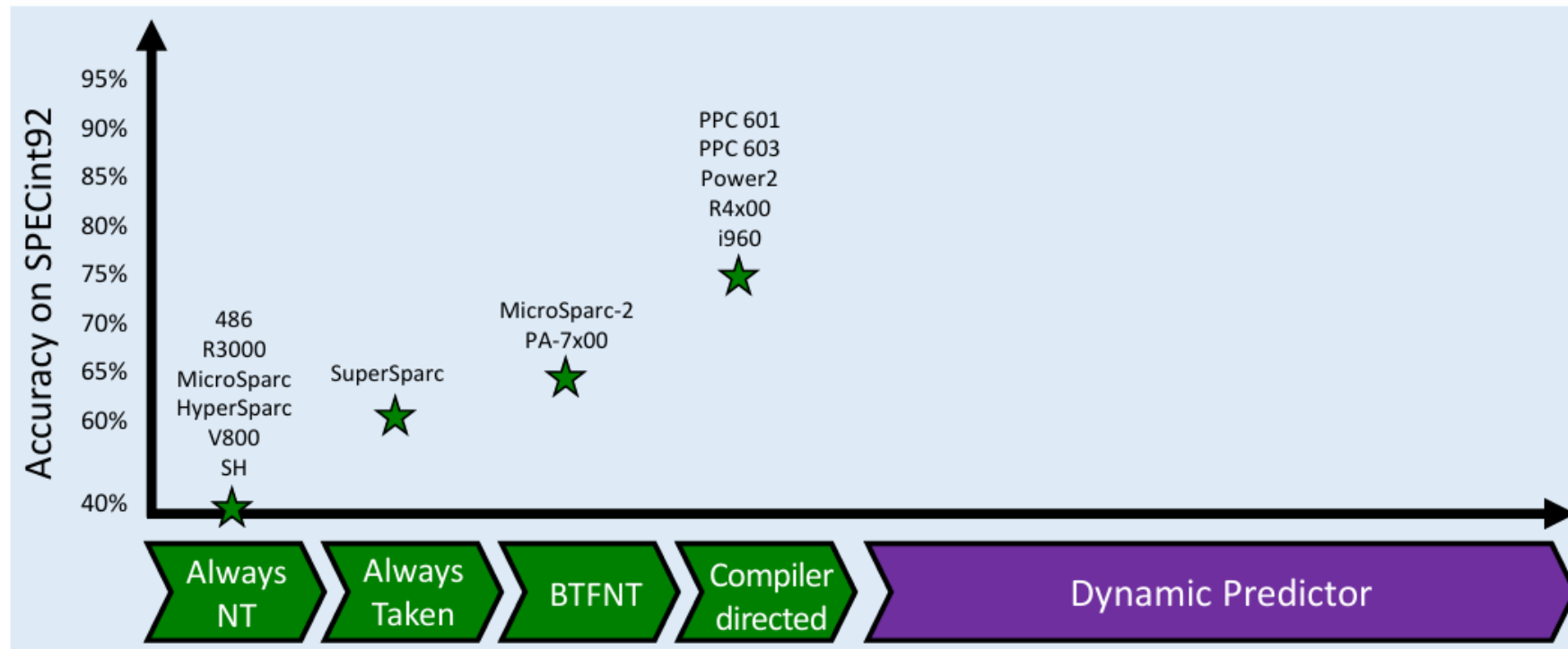
Dynamic Branch Prediction

1 Bit

Difference between Static and Dynamic Prediction

Dynamic Direction Prediction

- *Static* predicts any particular branch in the same way whenever it is encountered
- To achieve a greater accuracy, *dynamic* takes into account **run-time** info.
- The processor learns from its mistakes and changes its predictions to match the behavior of **each particular branch**

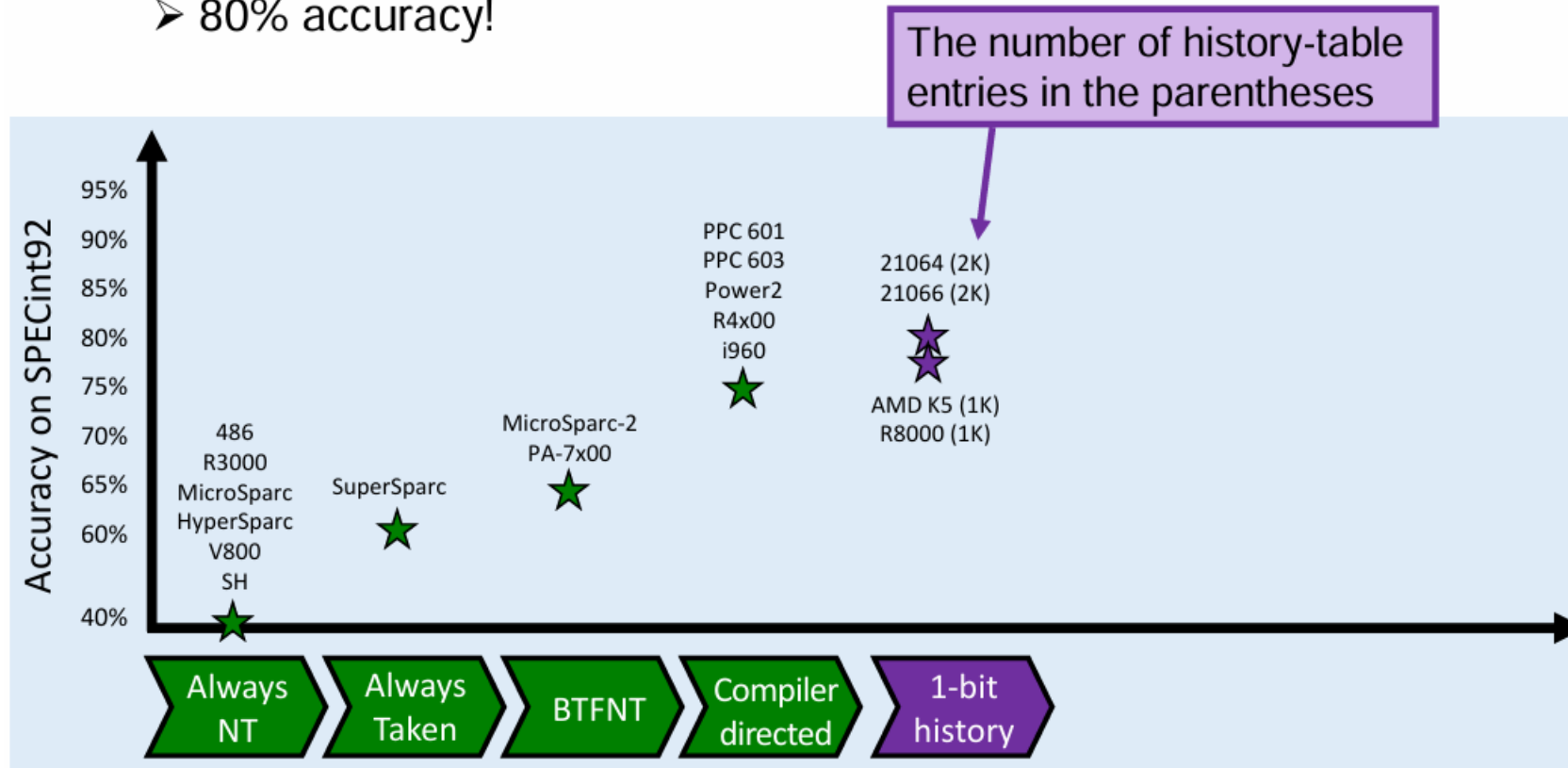


Using 1 Bit History in BHT

► Dynamic Direction Prediction

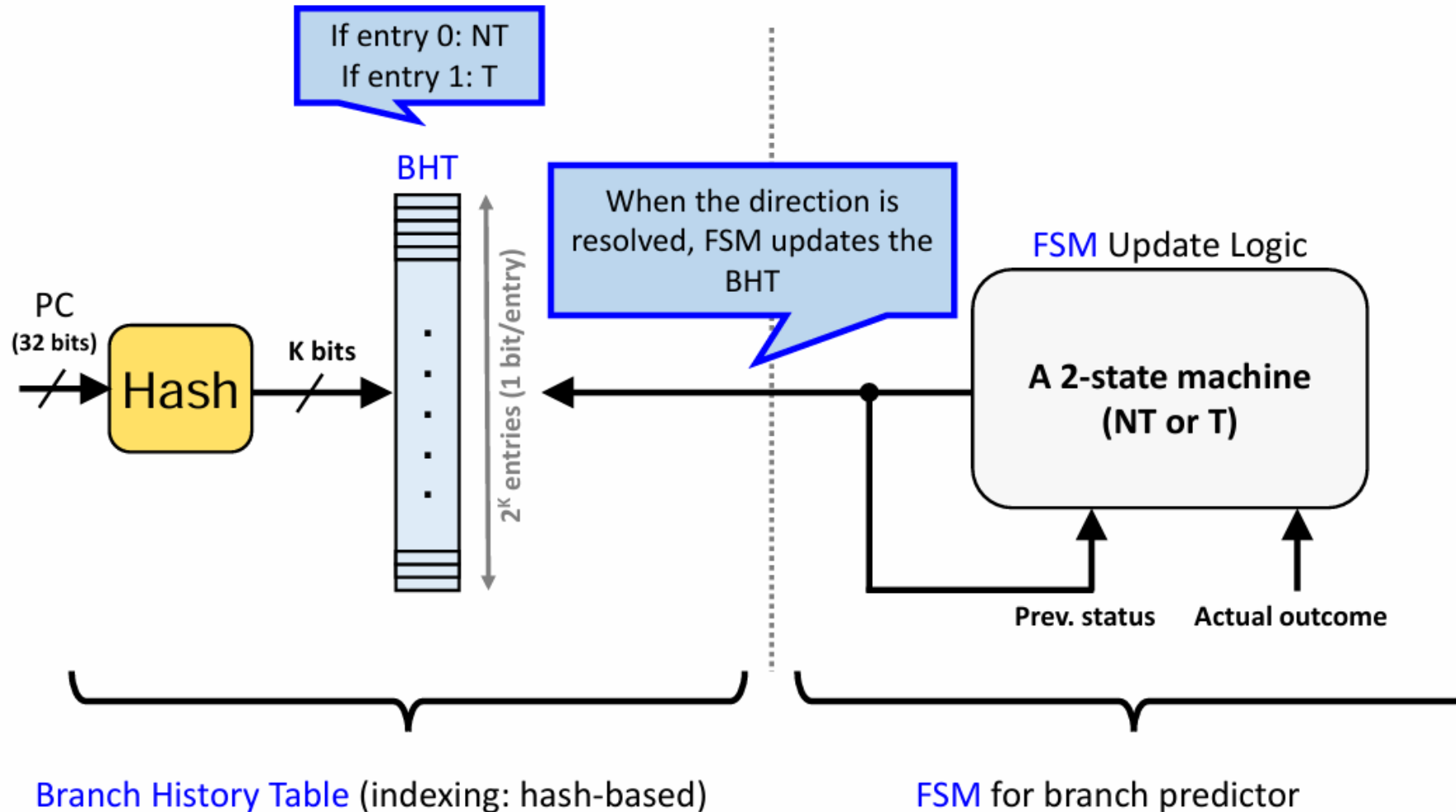
① 1-Bit History (Cont'd)

- Use **Branch History Table (BHT)** to keep a record of the previous branch behavior, allowing it to improve its predictions over time
- Maintains a single history bit for each branch
- 80% accuracy!

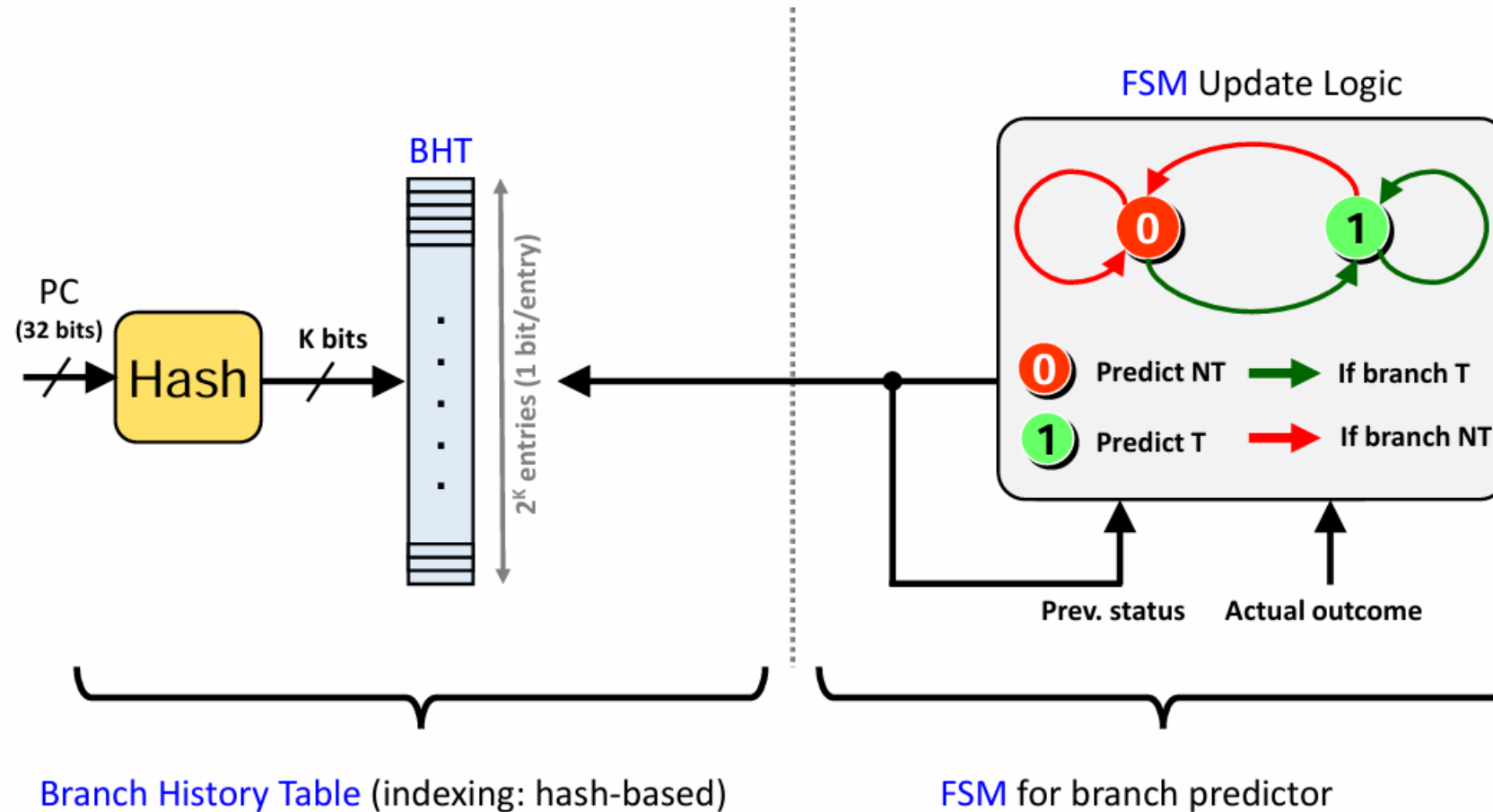


1 Bit Branch Predictor Architecture

► One-Bit Branch Predictor Architecture



► One-Bit Branch Predictor Architecture



Example 1: A loop using 1 Bit BHT

► Example 1: A loop using 1-bit BHT

Program A

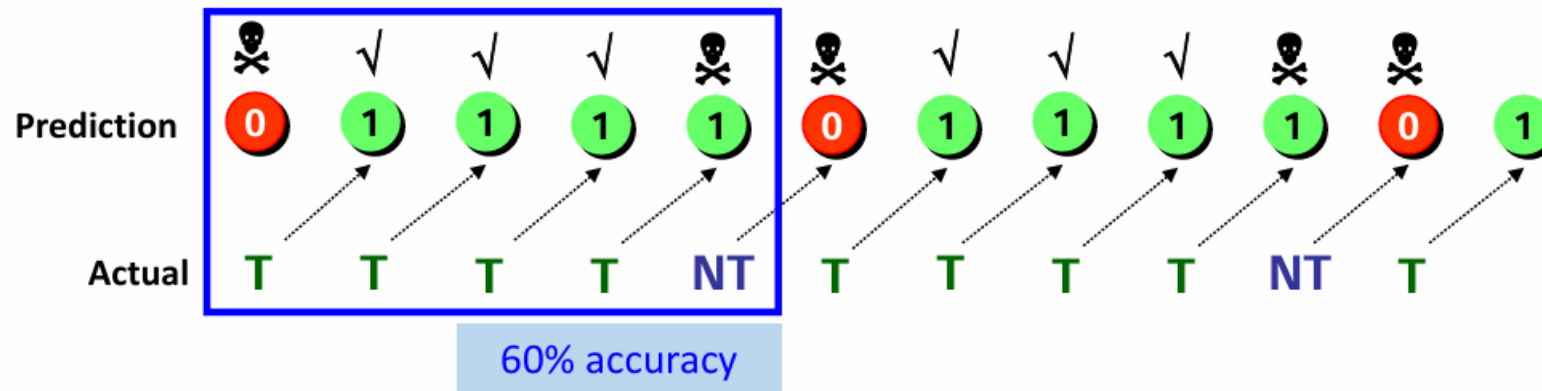
```
for (i=0; i<5; i++)
{
    ...
}
```



Assembly of Program A

```
addi r10, r0, 5
addi r1, r1, r0

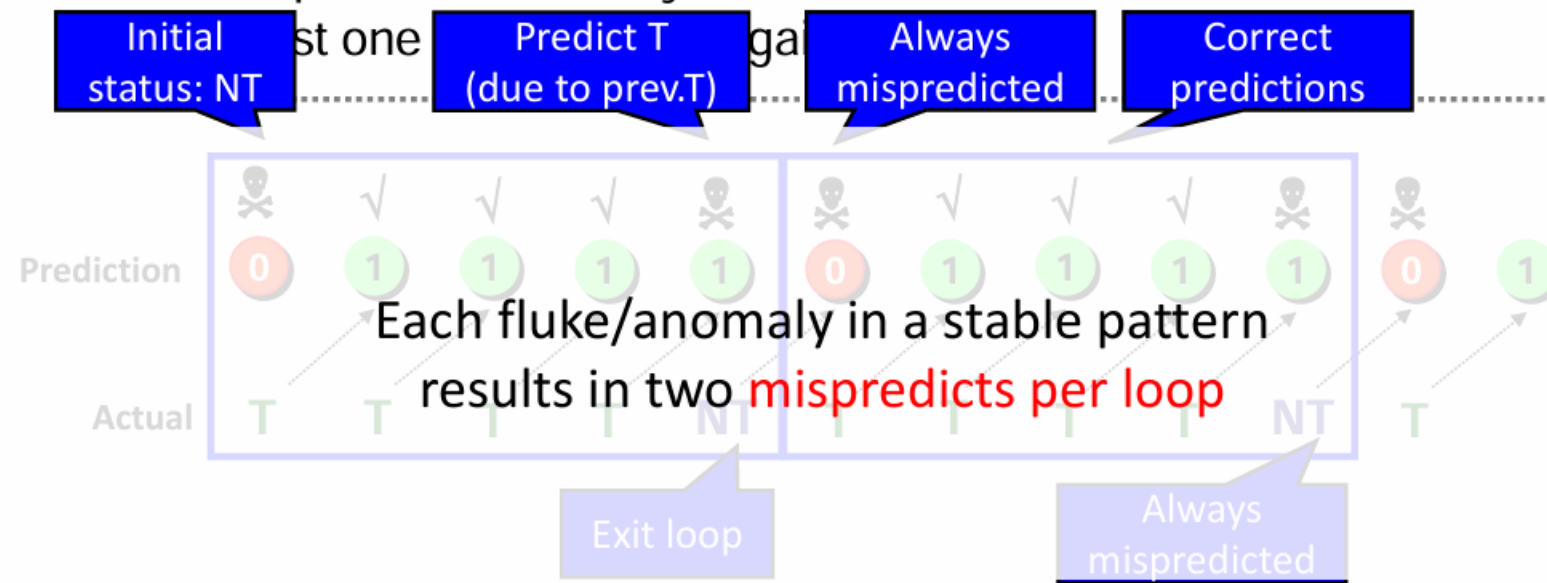
L1:
...
addi r1, r1, 1
bne r1, r10, L1
...
```



Challenge in 1 Bit Branch Predictor

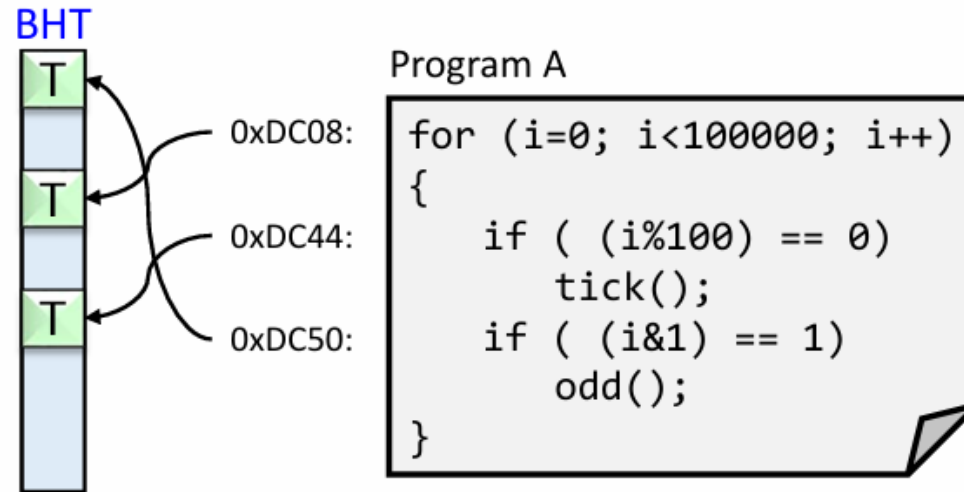
► **Challenge: The Bit Is Not Enough**

- Example: short loop (5 iterations)
 - Taken 4 times, then not taken once (exit loop)
 - First depends on the initial status of predictor
 - Not-taken mispredicted (was taken previously)
- Execute the same loop again
 - First always mispredicted (previous outcome was not taken)
 - Then 3 predicted correctly



Example2: Variant Branch Cases

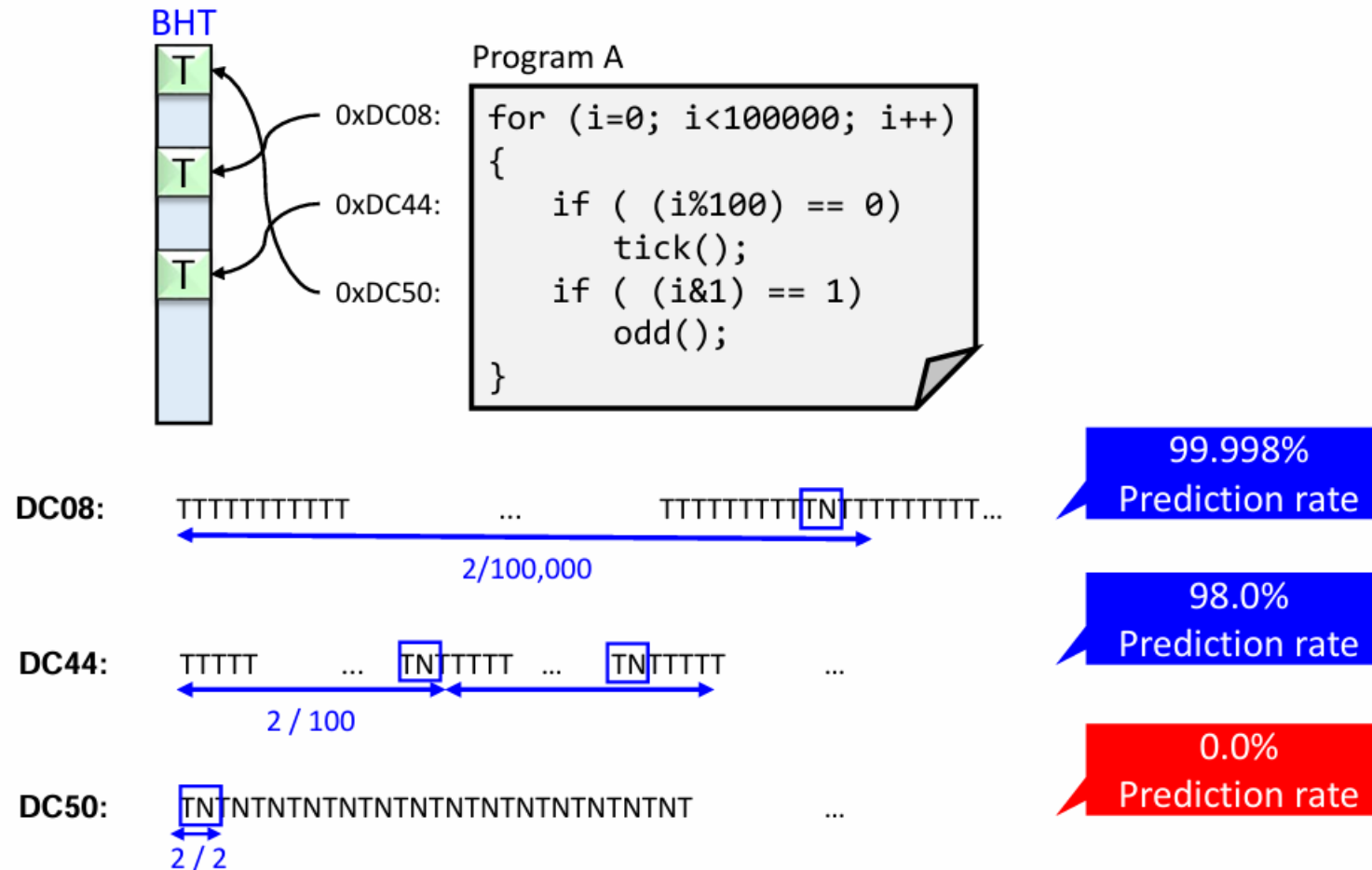
- Various branch cases based on length of T/NT iteration



Challenge of Multiple Branches in 1 Bit Predictor

Challenge: Not Work for Short Iter.

➤ How often is branch outcome $\neq$ previous outcome?



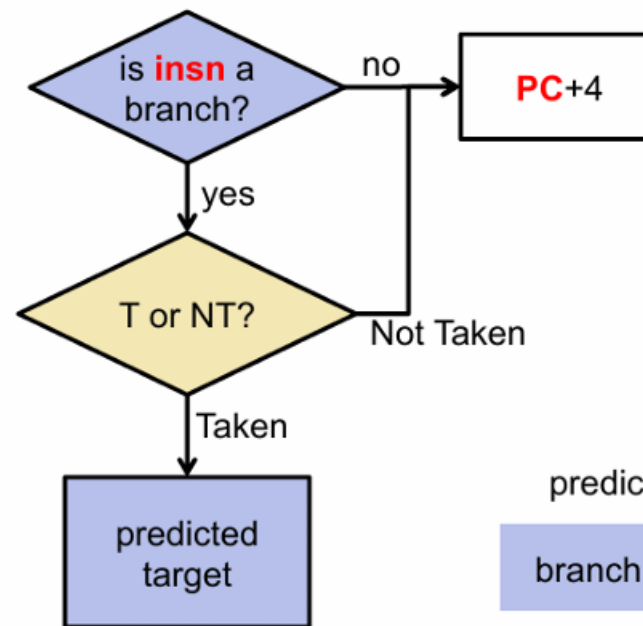
Dynamic Branch Prediction

2 Bit

How Dynamic Branch Prediction is Performed?

Dynamic Branch Prediction

- **Branch Prediction steps**



- Which **insn**'s behavior are we trying to predict?
- Where does **PC** come from?

prediction source:

branch target buffer

direction predictor

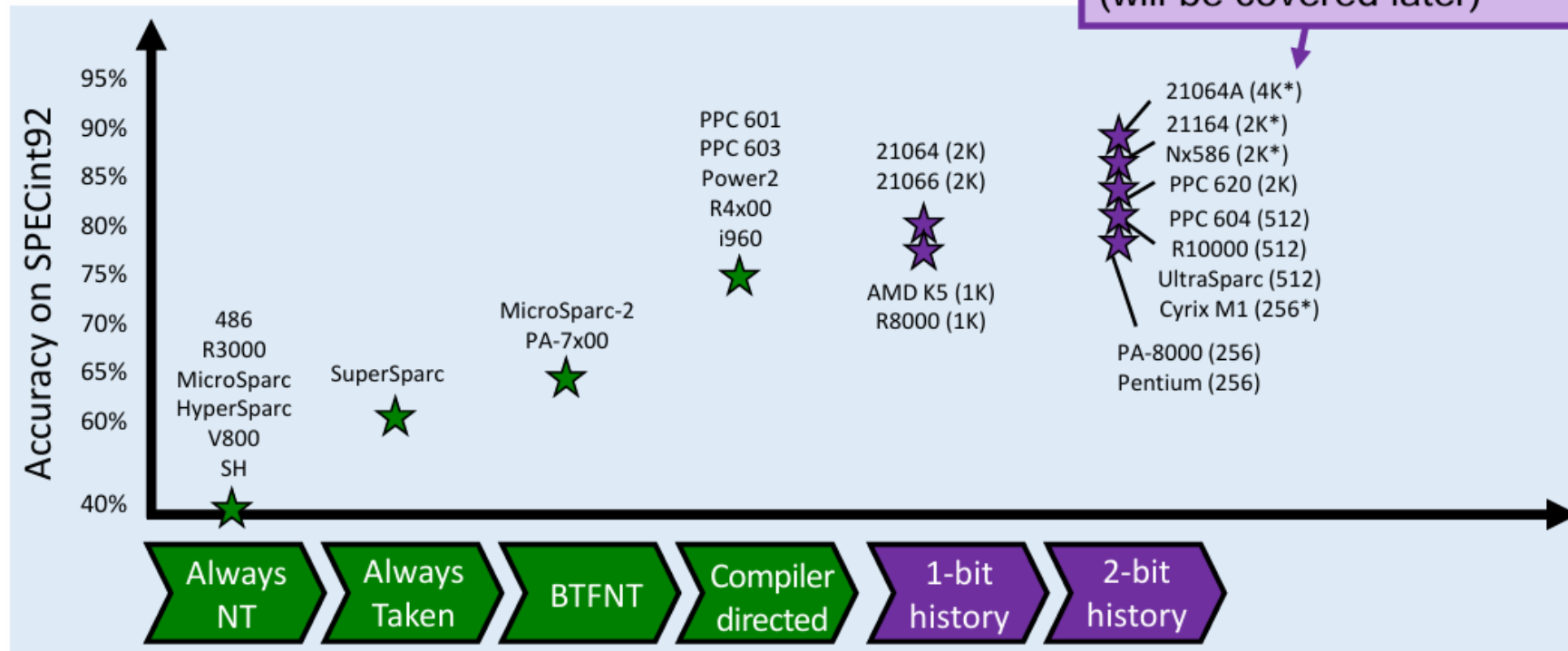
2 Bit History in BHT

➤ **Dynamic** Direction Prediction

② 2-Bit History

- Branch History Table (BHT) maintains **2 bit** per entry
- Now, FSM has four status (00, 01, 10, 11)
- For the best case, ~ 90% accuracy

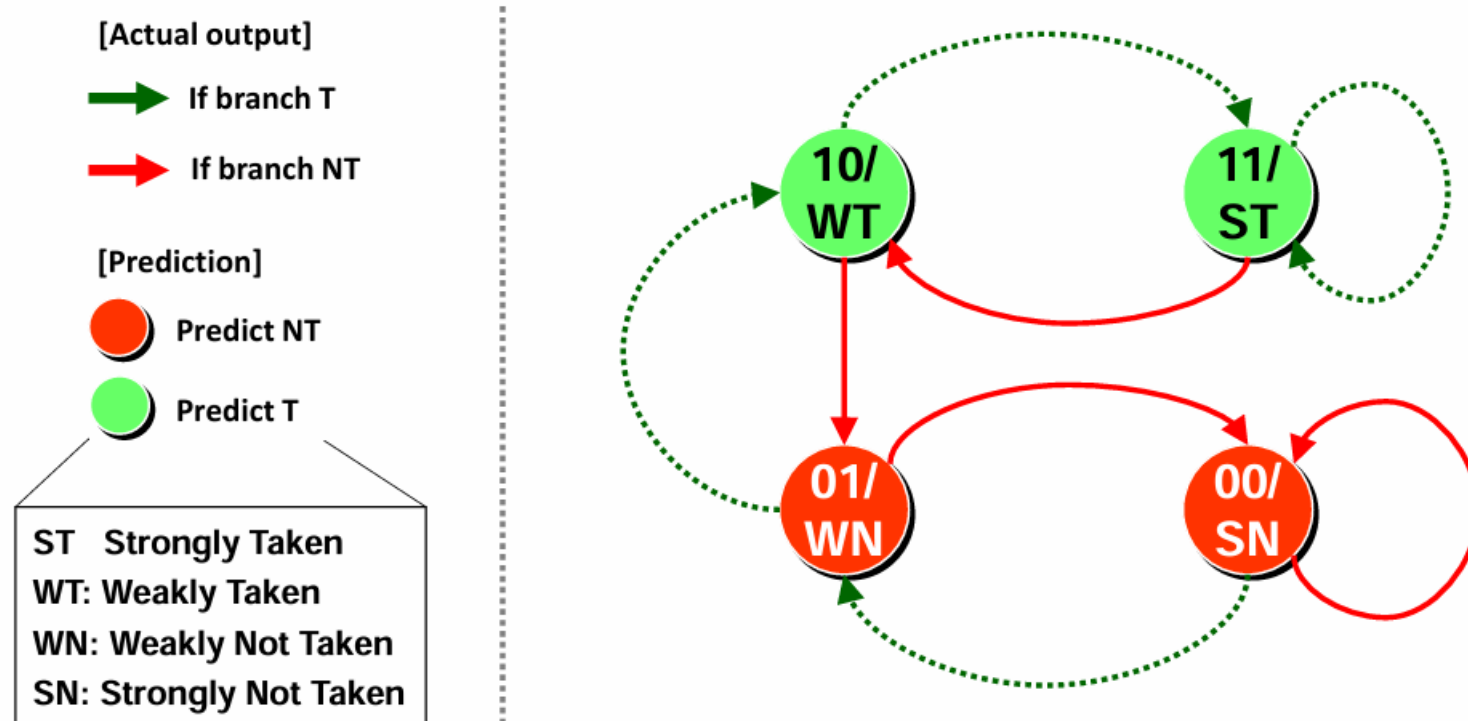
* means the target processor employs a return-address stack (will be covered later)



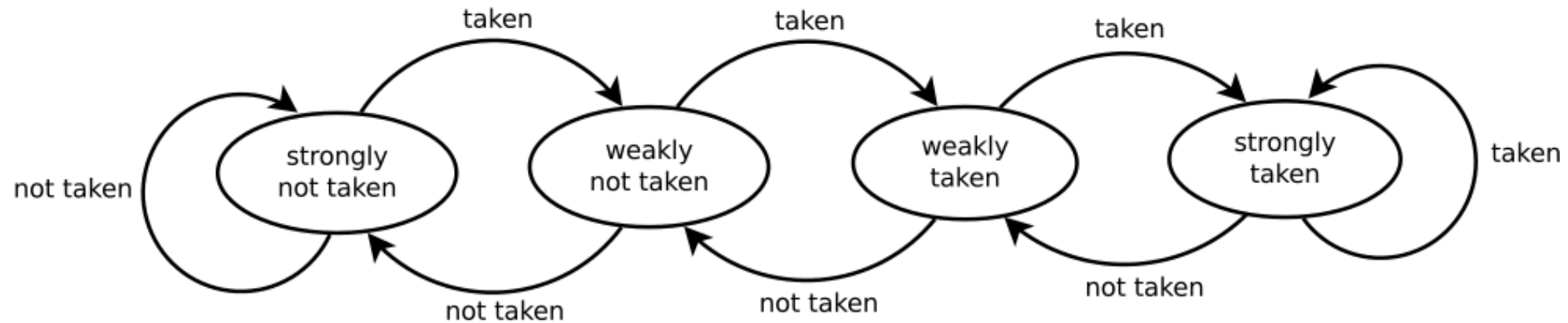
Basis of 2 Bit Branch Predictor

► **Idea: Two Bits are Better Than One**

- 2-bit Saturating Up/Down Counter Predictor
 - MSB: Direction bit
 - LSB: Hysteresis bit (concept: relaxation)



2 Bit Saturating Branch Predictor



- Consider a loop

- $i = 0$

- Loop:

- ...

- $i = i + 4$

- if $i < 400$ go to loop

- Branch Prediction trace

- What state do we start in?

- Assume something say SNT

i	State	Pred	Actual	Next State
0	SNT	NT	Taken	WNT
1	WNT	NT	Taken	WT
2	WT	T	Taken	ST
3	ST	T	Taken	ST

Example 1: 2 Bit Branch Predictor in a Loop

► Example 1: A loop using 2-bit BHT

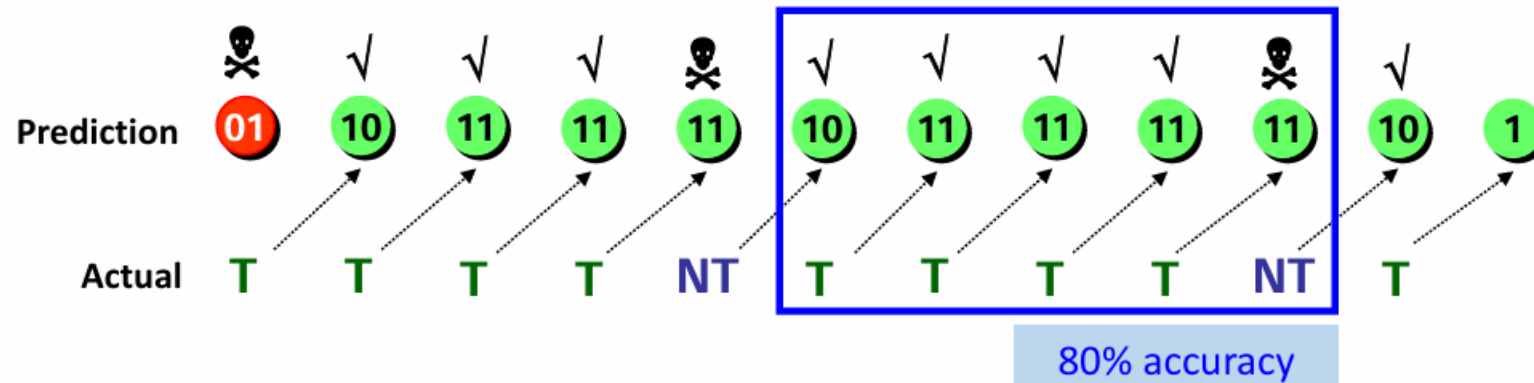
Program A

```
for (i=0; i<5; i++)
{
    ...
}
```

Assembly of Program A

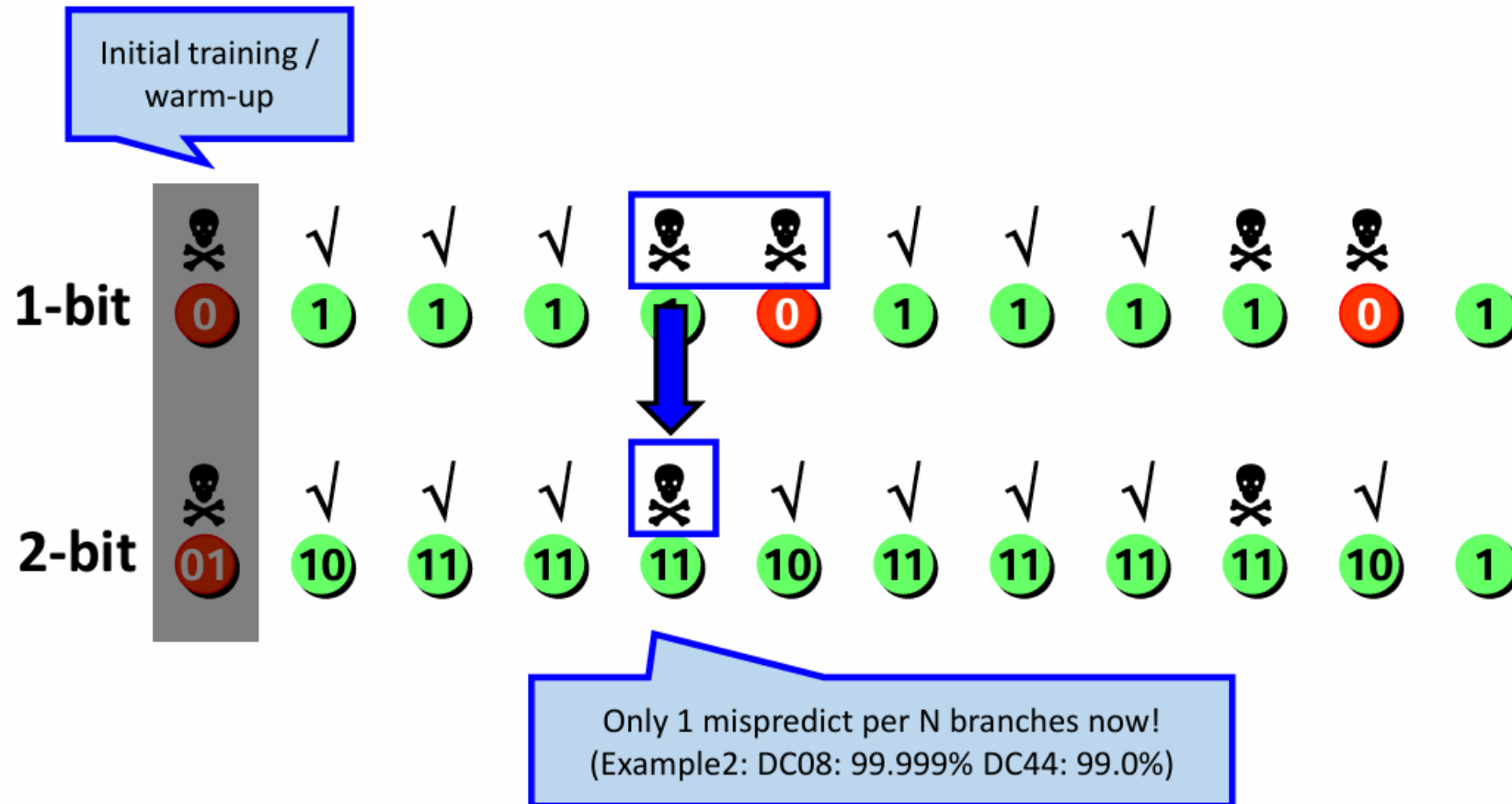
```
addi r10, r0, 5
addi r1, r1, r0

L1:
...
addi r1, r1, 1
bne r1, r10, L1
...
```



Compare 1 Bit vs 2 Bit History in BHT

► 1-bit History vs. 2-bit History (Example1)



Example of 1 Bit Branch Predictor

1-bit Branch Predictor

- 1-bit direction predictor

- Problem: **inner loop branch** below

```

for (i=0; i<100; i++)
  for (j=0; j<3; j++)
    // whatever
  
```

- The outcome is based on previous correct prediction
- Two “built-in” mis-predictions per inner loop iteration
- Branch predictor “changes its mind too quickly”

Time	State	Prediction	Outcome	Result?
1	N	N	T	Wrong
2	T	T	T	Correct
3	T	T	T	Correct
4	T	T	N	Wrong
5	N	N	T	Wrong
6	T	T	T	Correct
7	T	T	T	Correct
8	T	T	N	Wrong
9	N	N	T	Wrong
10	T	T	T	Correct
11	T	T	T	Correct
12	T	T	N	Wrong

6 incorrect prediction Result?

Example of 2 Bit Branch Predictor

2-bit (Bimodal) Branch Predictor

- **Two-bit saturating counters (2bc)**

- Replace each single-bit prediction
 - $(0, 1, 2, 3) = (N, n, t, T)$
- Adds “hysteresis”
 - Force predictor to mis-predict twice before “changing its mind”
 - One mis-predict each loop execution (rather than two)
 - + Fixes this pathology (which is not contrived) **5 incorrect prediction**

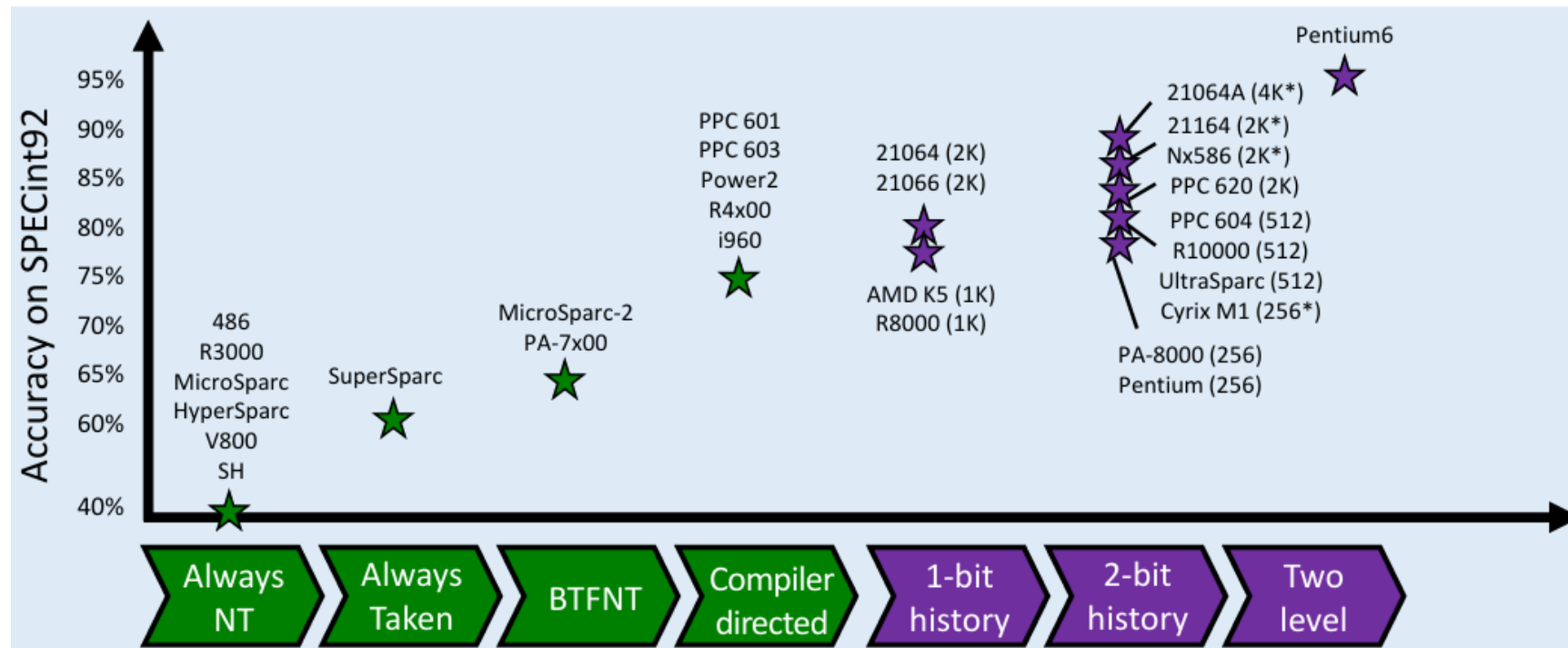
Time	State	Prediction	Outcome	Result?
1	N	N	T	Wrong
2	n	N	T	Wrong
3	t	T	T	Correct
4	T	T	N	Wrong
5	t	T	T	Correct
6	T	T	T	Correct
7	T	T	T	Correct
8	T	T	N	Wrong
9	t	T	T	Correct
10	T	T	T	Correct
11	T	T	T	Correct
12	T	T	N	Wrong

Dynamic Branch Prediction Correlated and Tournament

➤ **Dynamic** Direction Prediction

③ Two Level

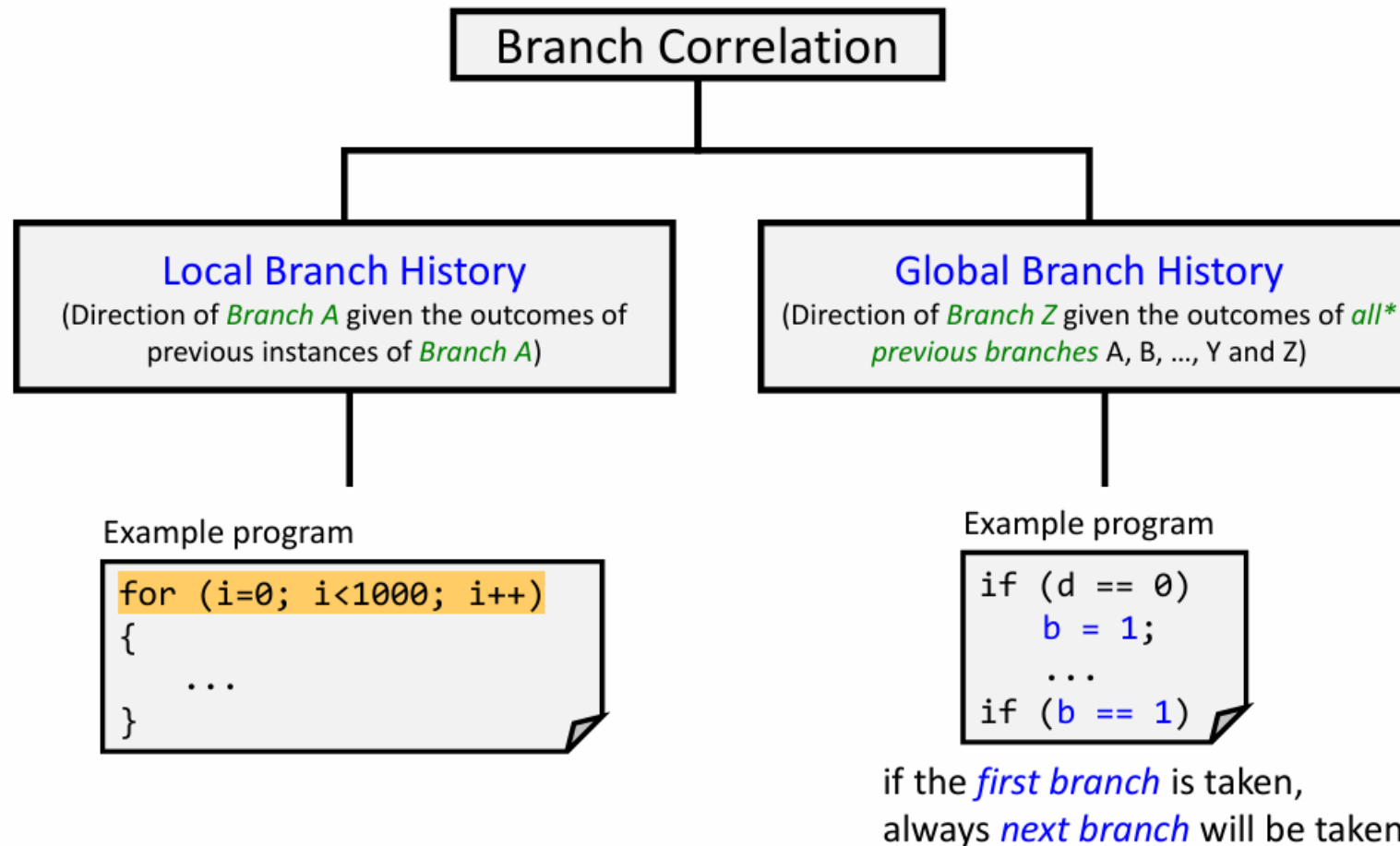
- Use **branch correlation**!
 - The outcome of a branch is often related to previous outcome(s)
- Use both Branch History Table (BHT) & **Pattern History Table** (PHT)
- 96% accuracy at the best!



Branch Correlation in BHT

Branch Correlations

- Can be categorized based on branch behaviors



Example of Global Correlation in Branches

► More Examples of Global Correlation

Testing same/similar conditions

```
void fill_foo(int* foo)
{
    if (foo != NULL)
        *foo = 23;
}

if (foo != NULL)
    fill_foo(&foo)
```

- The caller might test the argument for NULL; the callee can also test foo for NULL, again

Partial correlations

```
if (cond1)
{..}
if (cond1 && cond2)
{ ..}
```

- The first branch could examine **cond₁**, and the second branch can test **cond₁ && cond₂**
- If **cond₁** is false, then the second branch can be predicted as false

Multiple correlations

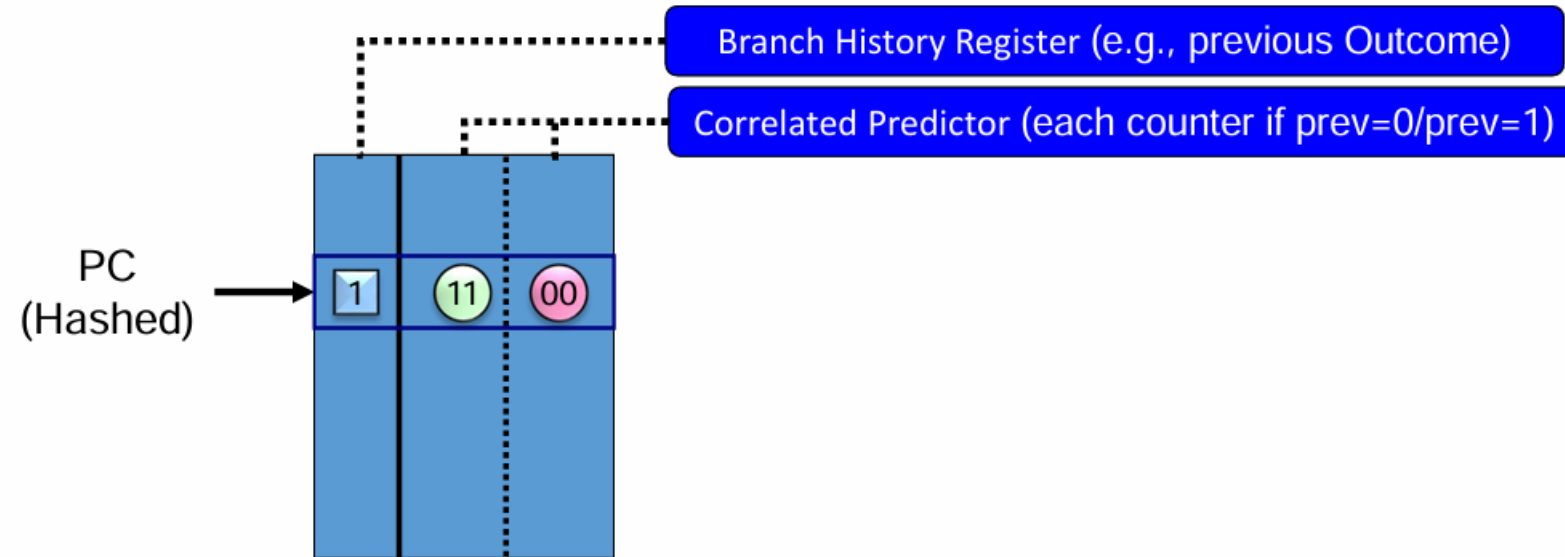
```
if (cond1)
{..}
if (cond2)
{..}
if (cond1 && cond2)
{ ..}
```

- The branch tests **cond₁**, and the second tests **cond₂**
- The third examines **cond₁ ⊕ cond₂**, which can always be predicted if the direction of first two branches is turned out

Pattern History Table in Correlated Predictor

Correlated Predictors

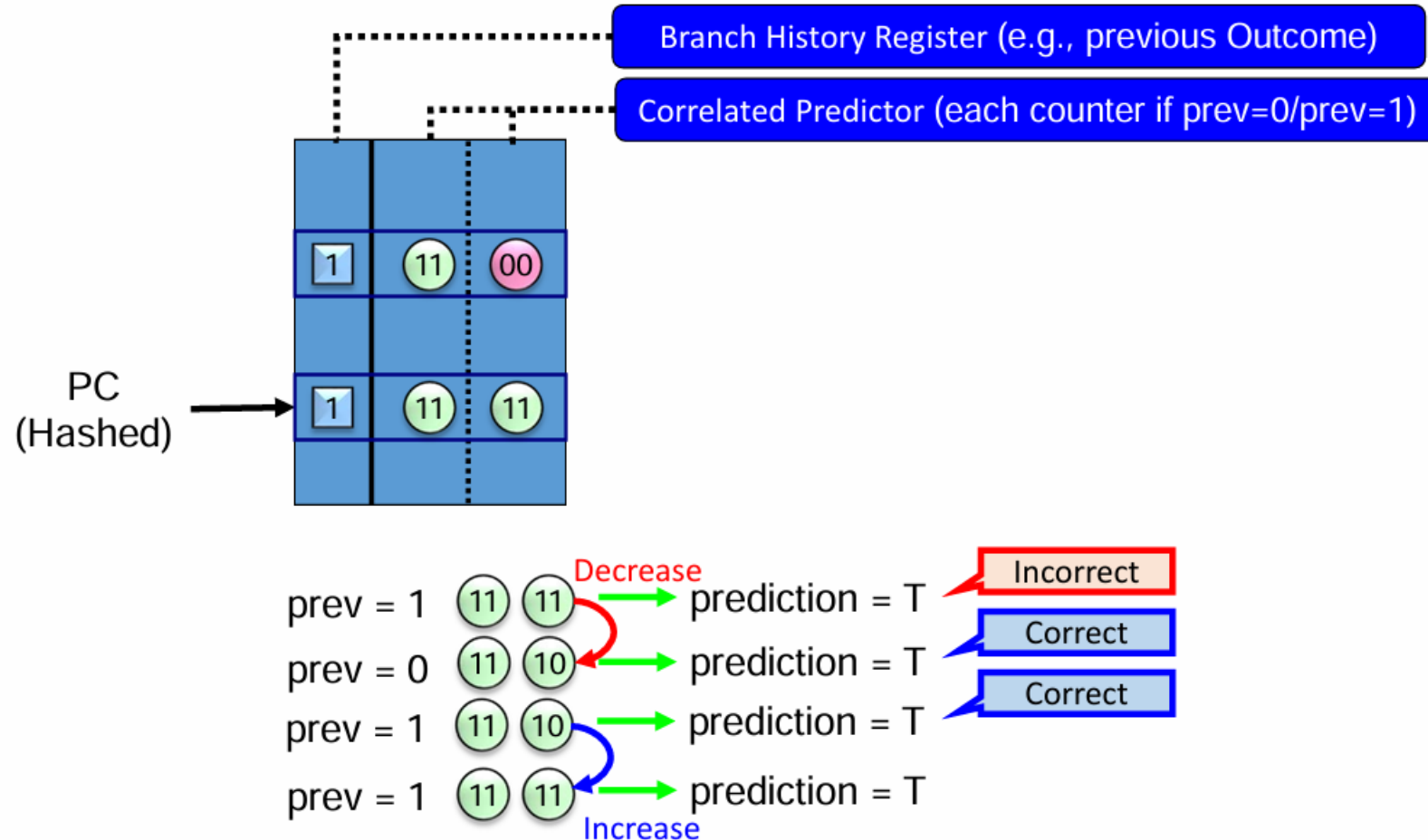
- Track the *history* of a branch by using Pattern History Table (PHT)



prev = 1	11	00	→ prediction = N	Correct
prev = 0	11	00	→ prediction = T	Correct
prev = 1	11	00	→ prediction = N	Correct
prev = 0	11	00	→ prediction = T	

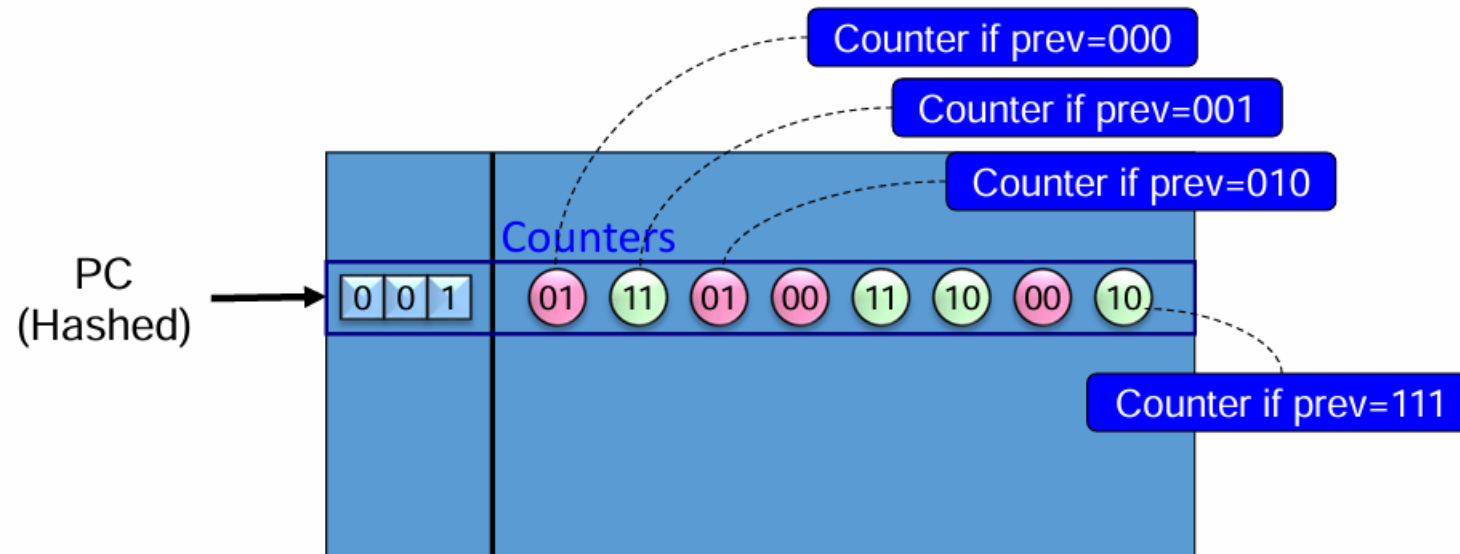
Correlated Predictors

- Track the *history* of a branch by using Pattern History Table (PHT)



Deeper History Covers More Patterns

- If Pattern History Table can maintain the last 3 outcomes..
- Question: what pattern has this predictor entry learned?
 - $001 \rightarrow 1; 011 \rightarrow 0; 110 \rightarrow 0; 100 \rightarrow 1$
 - $00110011001\dots (0011)^*$



► Design Policy of Correlated Predictors

- As you may already be aware, there are many implementation variations for the correlated predictor concept
 - ① The number of history register
 - 1 history register for all branches (**global**)
 - Table of history register, 1 for each branch (**private**)
 - Table of history registers, each shared by several branches (**shared**)
 - ② History length (size of history register)
 - ③ The number of pattern history tables
- The implementation will be affected by the actual **workload pattern**, the size of table limits at design time being aware of accuracy demands

Correlating Predictors (Global)

- 1- and 2-bit predictors exploit most recent history of the current branch
- Realization: Different branches maybe correlated!

```
if (a == 2)
    a = 0;
if (b == 2)
    b = 0;
if (a != b) {
    ...}
```

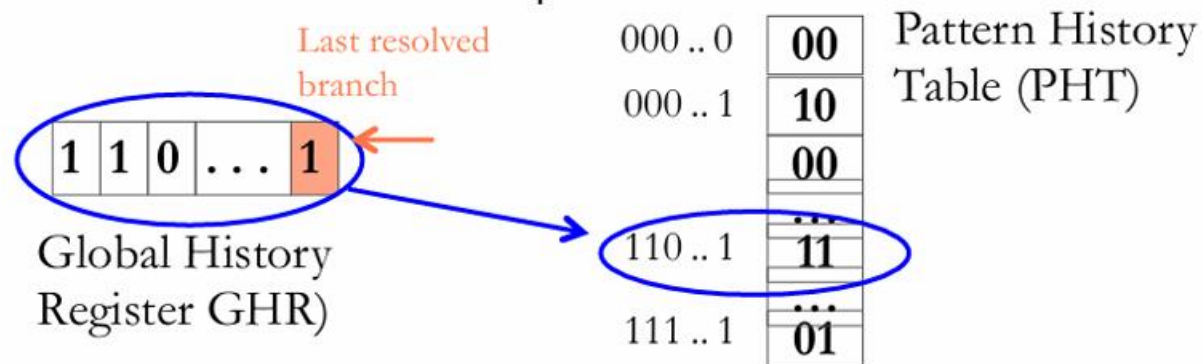
If both branches are taken,
the last branch definitely not taken

```
char s1 = "Bob"
...
if (s1 != NULL)
    reverse_str(s1);
reverse_str(char *s) {
    if (s1 == NULL)
        return;
    ...}
```

s1 definitely not Null
in this calling context

Global Two-level (Correlating) Predictor

- Prediction depends on the context of the branch
- Context: history (T/NT) of the last N branches
 - First level of the predictor
 - Implemented as a shift register
- Prediction: 2-bit saturating counters
 - Indexed with the “global” history
 - Second level of the predictor



Gshare History-Based Predictor

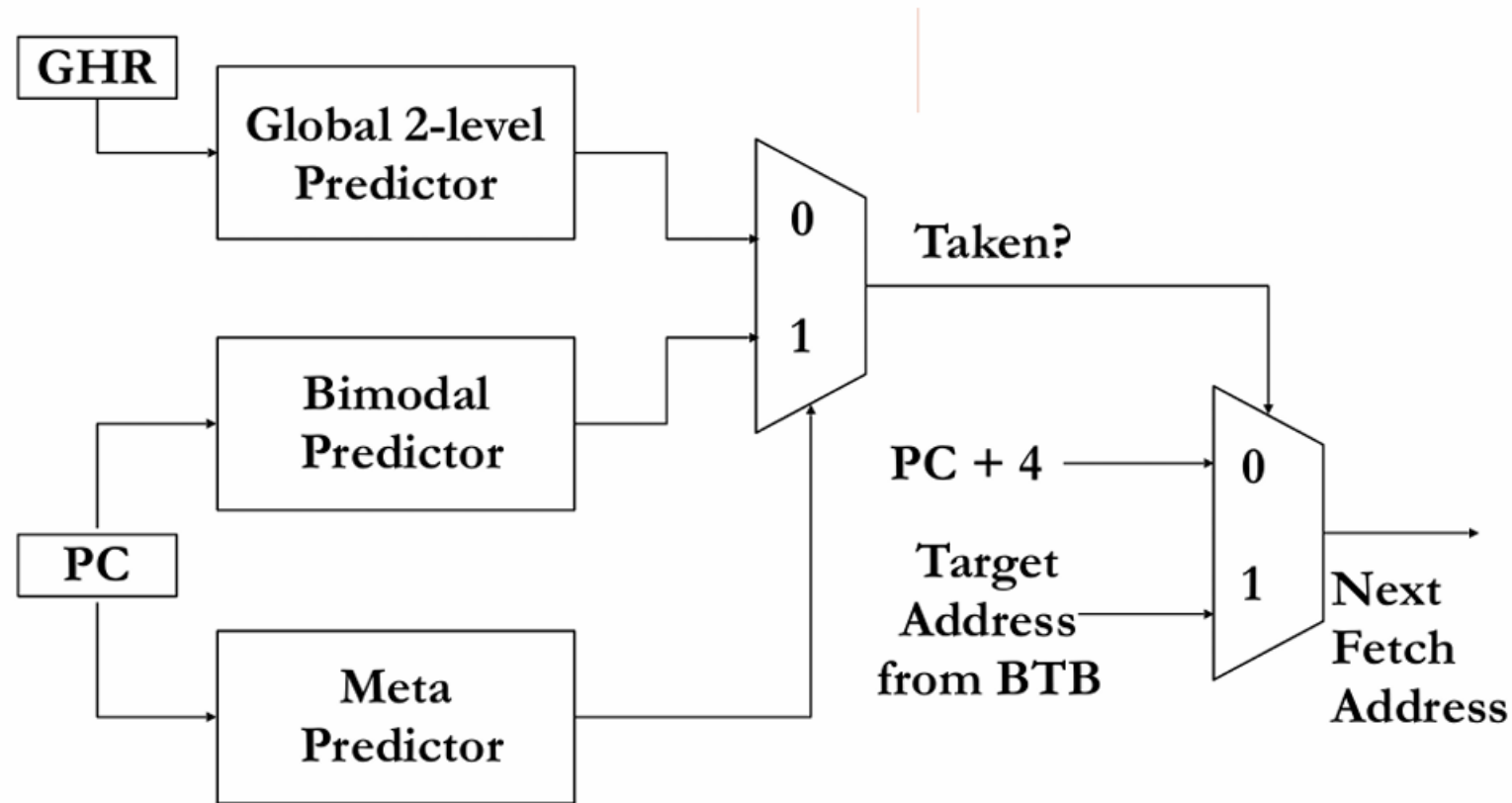
- Gshare working example
 - Assume program has one branch
 - BHT: one 1-bit DRIP entry
 - 3BHR**: last 3 branch outcomes
 - Train counter, **and** update BHR after each branch

Time	State	BHR	Prediction	Outcome	Result?
1	N	NNN	N	T	wrong
2	N	NNT	N	T	wrong
3	N	NTT	N	T	wrong
4	N	TTT	N	N	correct
5	N	TTN	N	T	wrong
6	N	TNT	N	T	wrong
7	T	NTT	T	T	correct
8	N	TTT	N	N	correct
9	T	TTN	T	T	correct
10	T	TNT	T	T	correct
11	T	NTT	T	T	correct
12	N	TTT	N	N	correct

Tournament Predictor

- Most branches are biased (e.g. 99% Taken)
- Filter the biased branches with a simple predictor (e.g. Bimodal)
- Predict the hard branches with the Global 2-level predictor
- Use a meta-predictor to choose a different predictor
- The meta-predictor is a PHT of 2-bit saturating counters

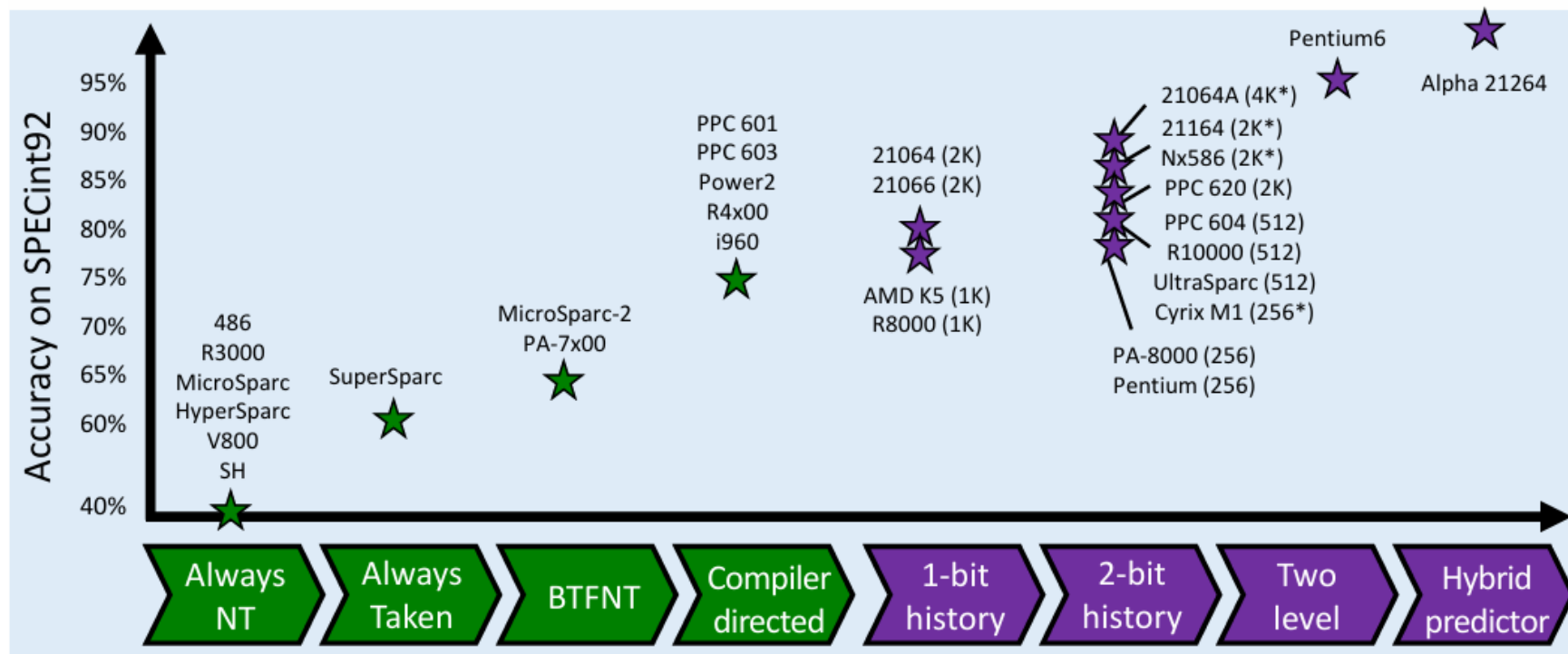
Tournament Predictor



➤ **Dynamic Direction Prediction**

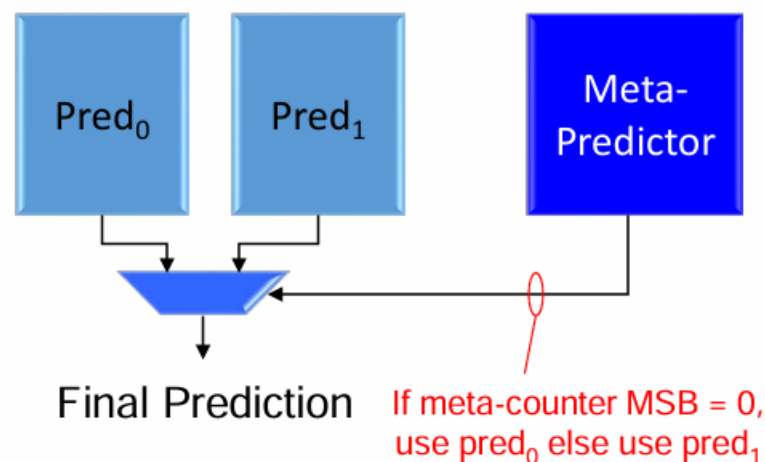
④ Hybrid Predictor (a.k.a Tournament Predictor)

- No predictor is clearly the best
 - Different branches exhibit different behaviors
- Idea: let's have a predictor to **predict** which **predictor** will predict better
- 97% accuracy at the best!



► Tournament Hybrid Predictor

- Combining two or more branch predictors
- Example:
 - Predictor0: local, per-branch prediction, which are accessed by the PC
 - Predictor1: correlated prediction based on the last "m" branches accessed by the global history
 - Meta-predictor of which had been the best predictor for this branch (table of 2-/3-bit counters)



Pred 0	Pred 1	Meta Update
x	x	---
x	✓	Inc
✓	x	Dec
✓	✓	---

FSM of meta-predictor

Predict Branch Target Address Design and Architecture

Predict Target Address of Branch

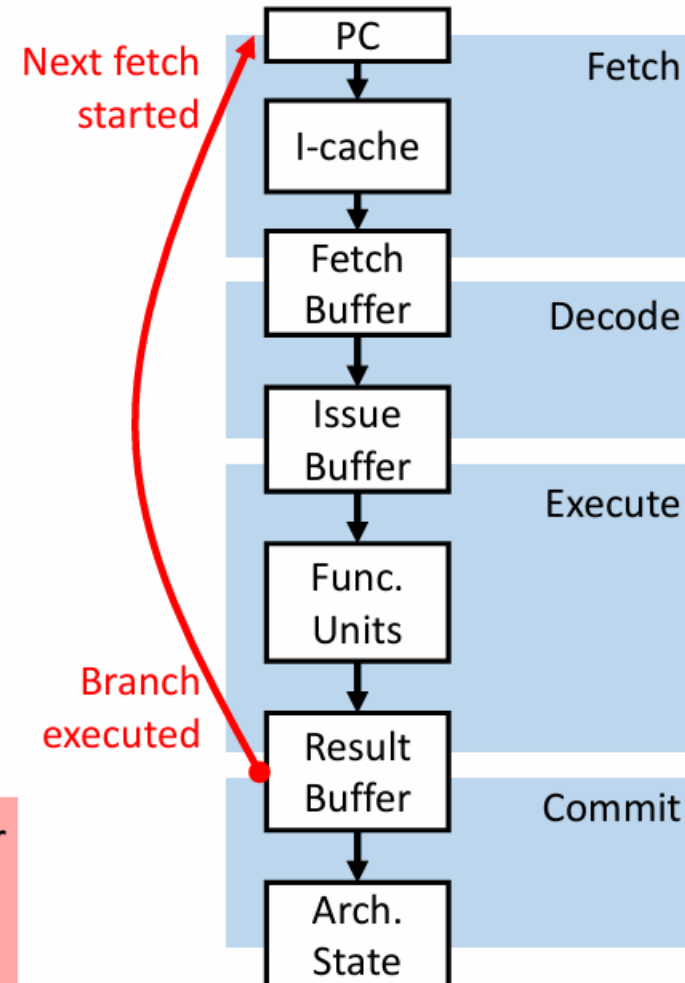
Let's Predict Branch Target Address

IF stage: needs to know the fetch address per cycle

ID stage: even easily-computed branch targets need to wait until the instruction decoded or direction predicted

EXEC stage: for some branches (e.g, indirect) target known only after EX stage, which is way too **late**

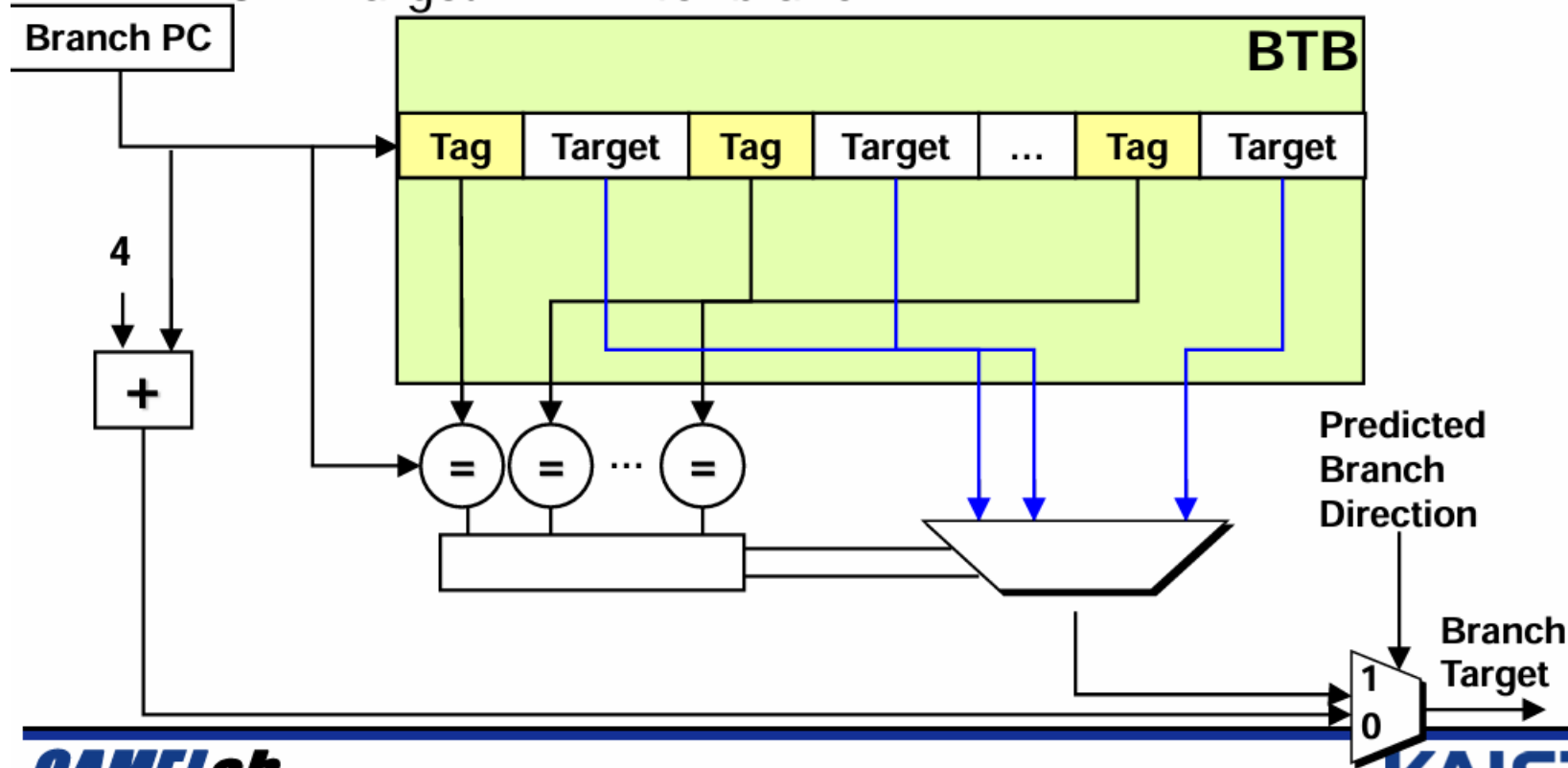
So, we have a quick-and-dirty predictor for the target that only needs the address of the branch instruction



Branch Target Buffer

► **Solution: Branch Target Buffer (BTB)**

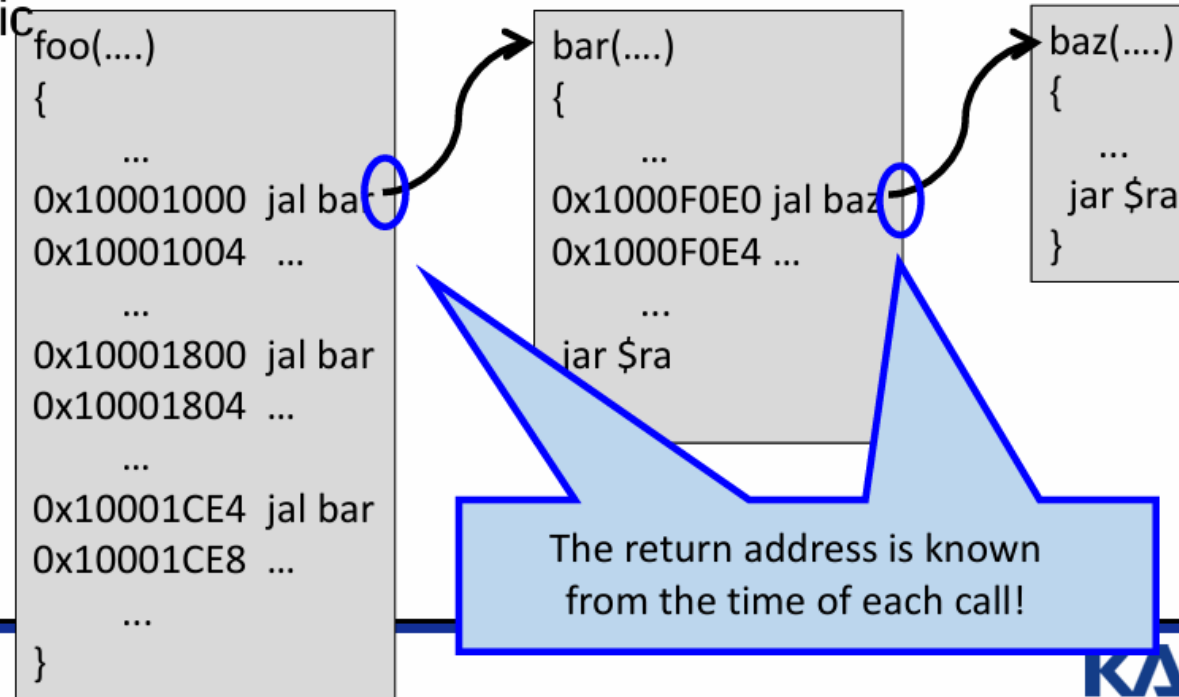
- A cache-like storage that records branch addresses and associated targets (indexed by the instruction address)
 - Why tag? – (Hint: desiring 1 cycle BTB lookup)
- If there is a **hit** in BTB for branch predicted taken:
 - $PC \leftarrow \text{Target in BTB for branch}$



Challenge in Function Return

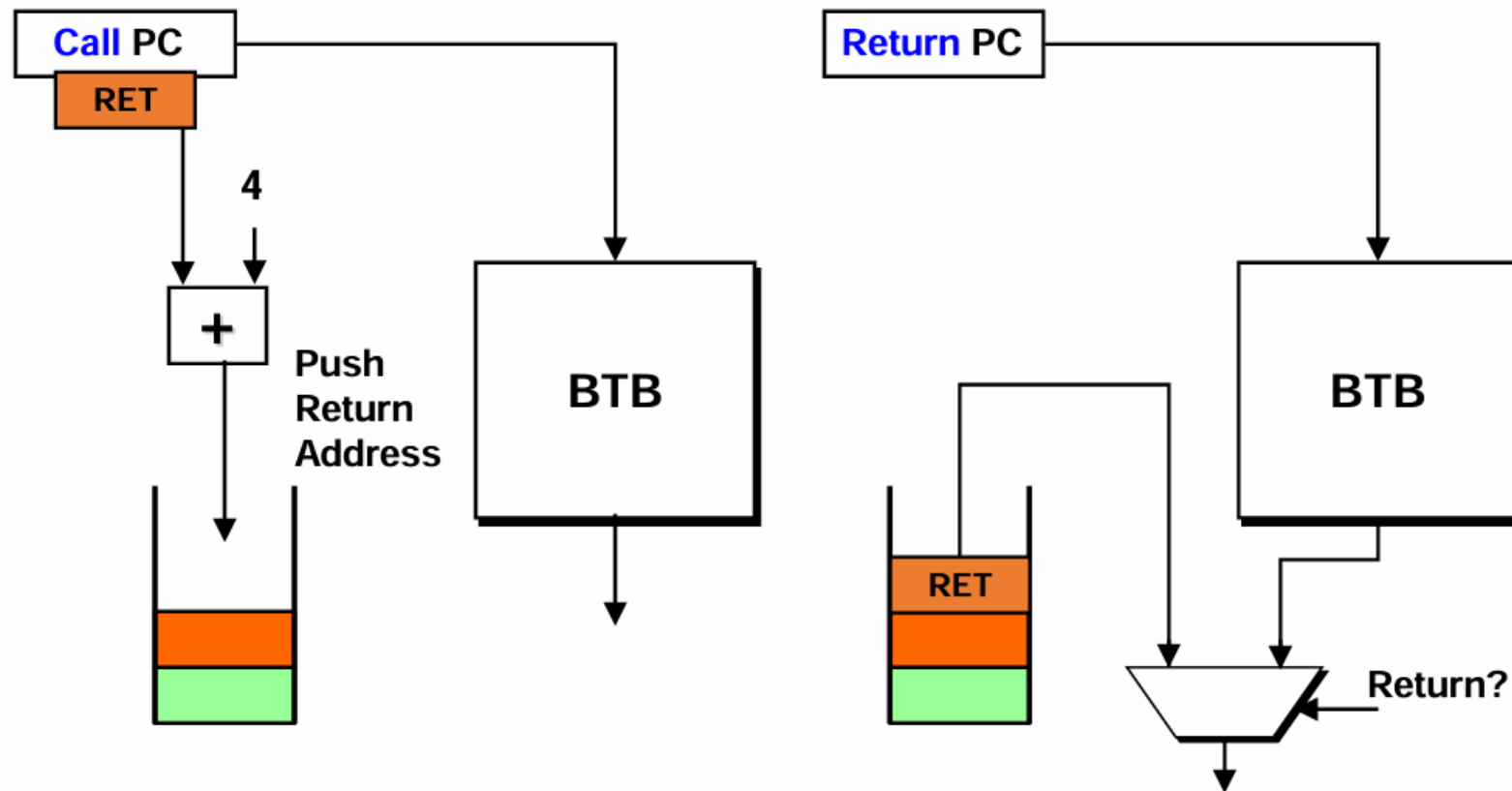
Challenge: Function Returns

- Function *returns* are frequent, yet
 - Address is difficult to compute (have to wait until EX stage done to know it)
 - Address is difficult to predict with BTB (function can be called from multiple places)
- But, a return address is in practice simple to predict if it's aware of ins. semantic



► **Solution:** Return Address Stack (RAS)

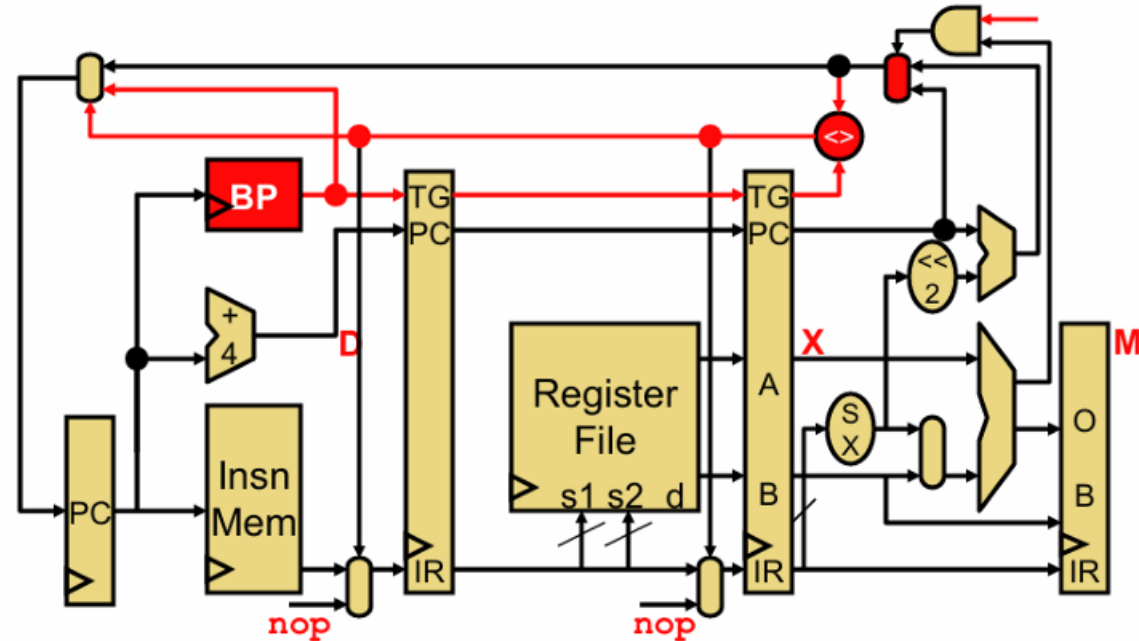
- Call pushes return address into the RAS
- When a return instruction decoded, pop the predicted return address from RAS



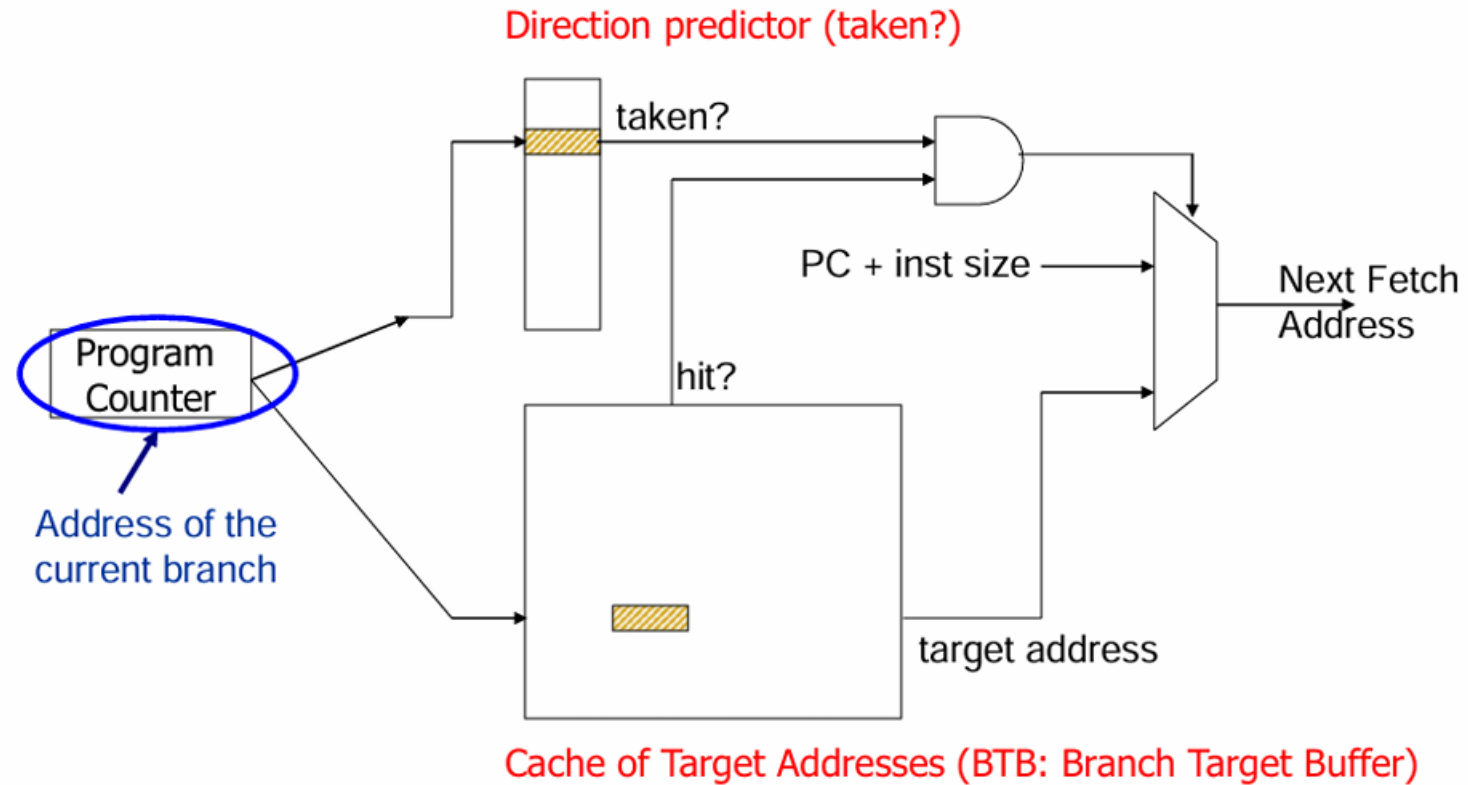
Dynamic Branch Prediction

- **Dynamic branch prediction**

- Hardware guesses outcome
- Start fetching from guessed address
- Flush on **mis-prediction**



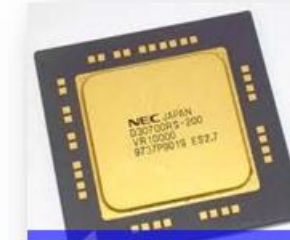
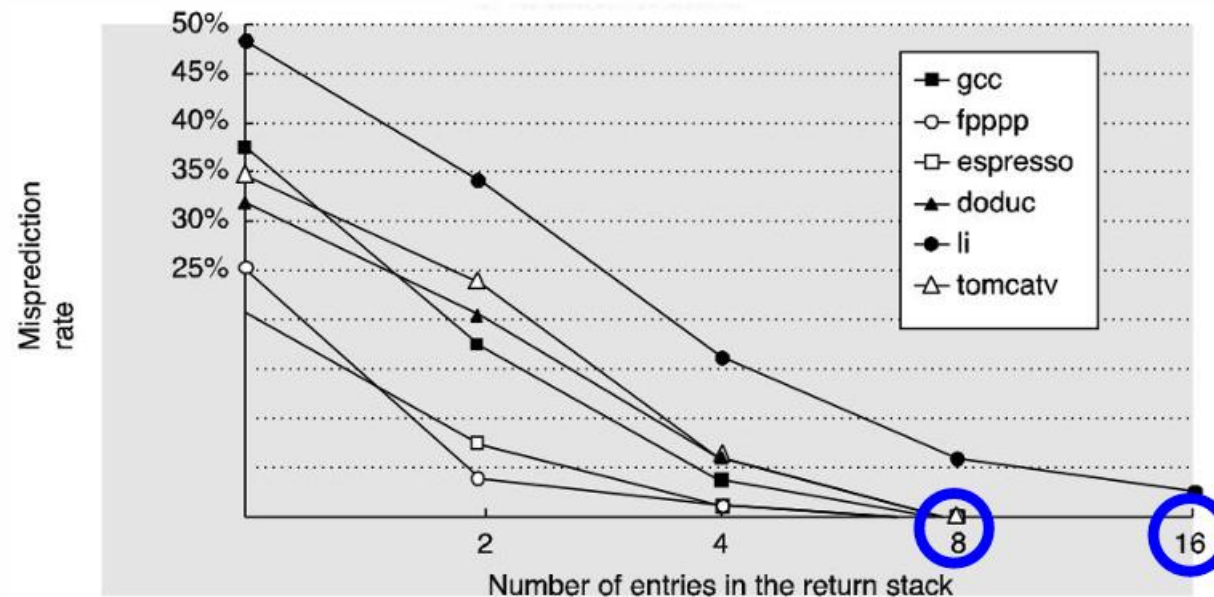
Dynamic Branch Prediction



RAS used in many processors

► **Solution:** Return Address Stack (RAS)

- Accurate prediction even with **small RAS**
 - **8 ~ 16** number of return stack entries is good enough!



MIPS R10K
(1-entry)

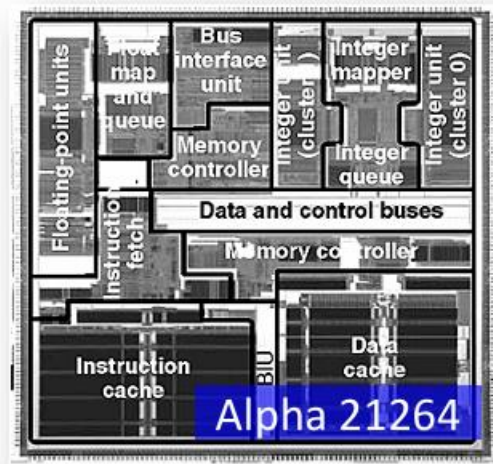


DEC Alpha
21164 (#12)

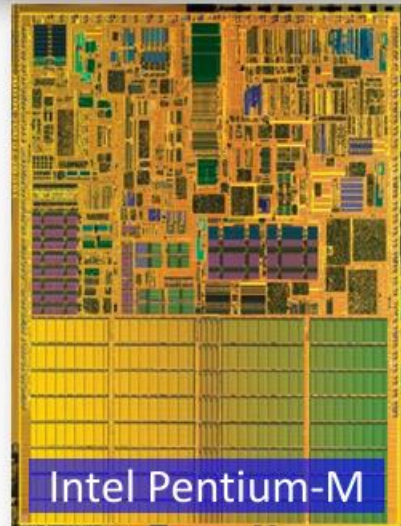


Intel P3 (#16)

Examples in Real μ Processor Architecture



- Highly speculative O3 Super-scalar
- Using a *hybrid predictor* that combines local & global history components with a meta-predictor (selector)



- Hybrid predictor that combines local & global history predictors, but uses tag-based selection mechanism
 - RAS/BTB is implemented, including a special component for indirect branches
- Common for object-oriented code (*vtables*)

► Examples of Branch Correlations

Related condition variable1

Example A

```
if (d == 0)
    b = 1;
...
if (b == 1)
```

Related condition variable2

Example B

```
if (x==2) /* branch 1 */
    x=0;
if (y==2) /* branch 2 */
    y=0;
if (x!=y) /* branch 3 */
    do this; else do that;
```

- Example A: if the *first branch* is taken, always *next branch* will be taken
 - Example B: if *branches 1 and 2* are taken, *branch 3* won't be taken
- ➔ Branch correlation based prediction could be done by keeping a history of the past "m" branches

Takeaway Questions

- Assume that
 - **Branch: 20%**, load: 20%, store: 10%, other: 50%
 - Say, 75% of branches are taken
 - What is the CPI?
- Assume that
 - **Branch: 20%**, load: 20%, store: 10%, other: 50%
 - Say, 75% of branches are taken
 - What is the CPI?
 - **$CPI = 1 + 20\% * 75\% * 2 = 1.3$**
 - **Branches cause 30% slowdown**
 - Worse with deeper pipelines, why?
 - Can we do better than assuming branch is not taken?

Branch Target Buffer

- Branch predictors tell whether the branch will be taken or not, but they say nothing about the target of the branch
- To resolve a branch early we need to know both the outcome and the target
- Solution: store the likely target of the branch in a table (cache) indexed by the branch PC → **BTB**
- Usually **BTB** is accessed in the IF stage and the branch predictor is accessed later in the ID stage

Handling Complex EX Stage with Multiple cycles

Pipeline Architecture with Multiple Execution

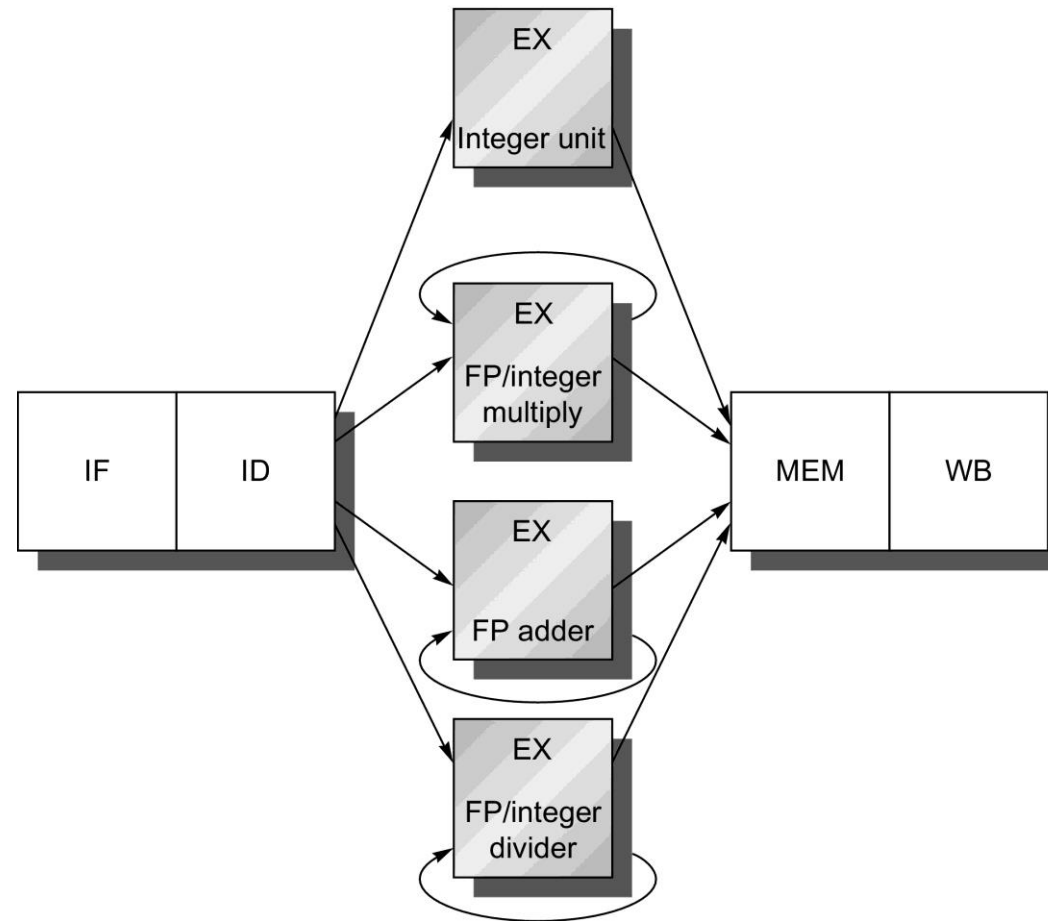


Figure C.28 The RISC V pipeline with three additional unpipelined, floating-point, functional units. Because only one instruction issues on every clock cycle, all instructions go through the standard pipeline for integer operations. The FP operations simply loop when they reach the EX stage. After they have finished the EX stage, they proceed to MEM and WB to complete execution.

Latency of Different EX Units



Functional unit	Latency	Initiation interval
Integer ALU	0	1
Data memory (integer and FP loads)	1	1
FP add	3	1
FP multiply (also integer multiply)	6	1
FP divide (also integer divide)	24	25

Figure C.29 Latencies and initiation intervals for functional units.

Multiple Outstanding Operations in Pipeline

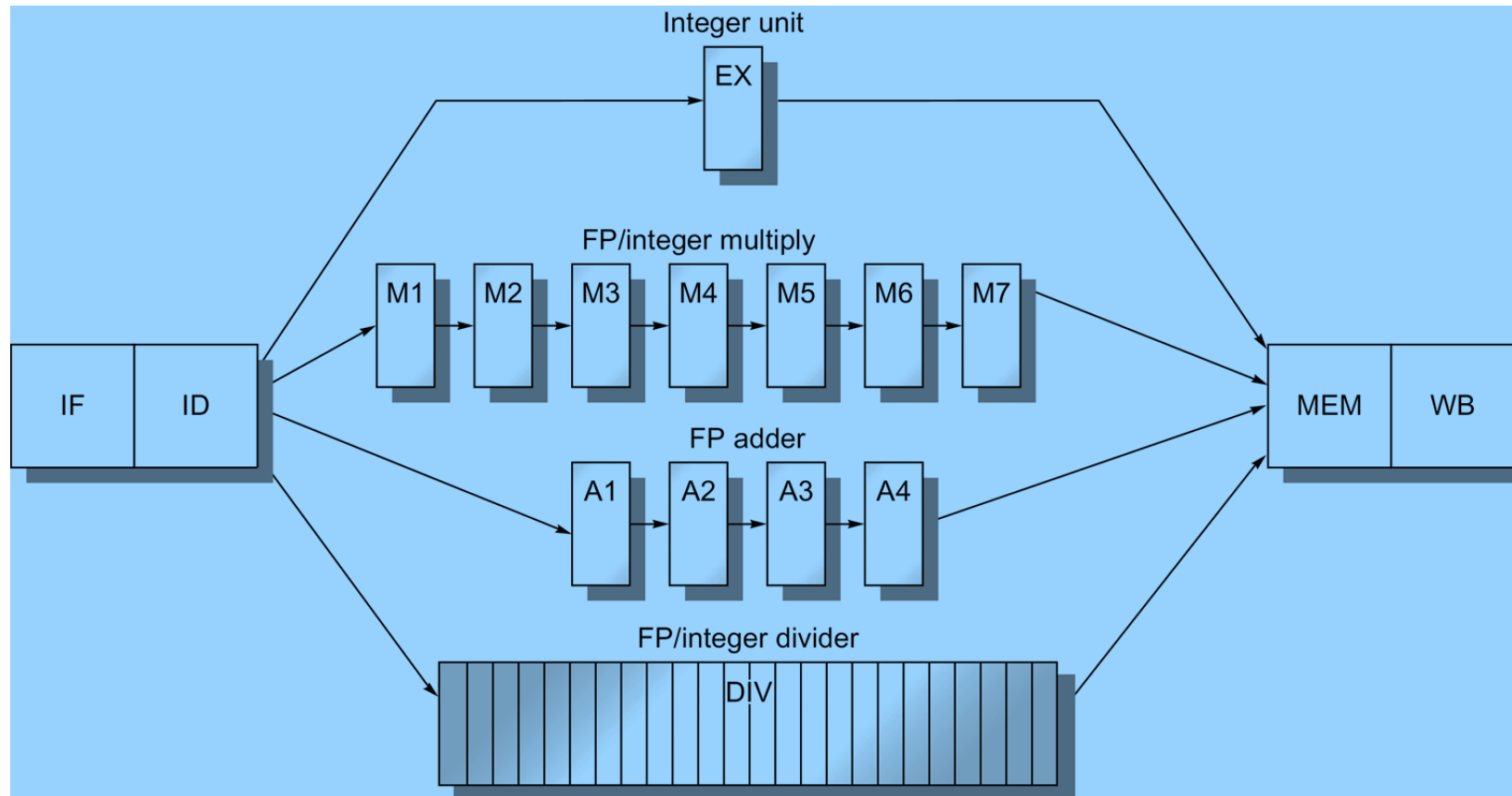


Figure C.30 A pipeline that supports multiple outstanding FP operations. The FP multiplier and adder are fully pipelined and have a depth of seven and four stages, respectively. The FP divider is not pipelined, but requires 24 clock cycles to complete. The latency in instructions between the issue of an FP operation and the use of the result of that operation without incurring a RAW stall is determined by the number of cycles spent in the execution stages. For example, the fourth instruction after an FP add can use the result of the FP add. For integer ALU operations, the depth of the execution pipeline is always one and the next instruction can use the results.